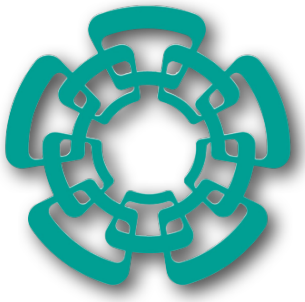
Cinvestav

Department of Physics



AI-driven Prediction of Circular Dichroism Spectra of Proteins.

M.Sc. thesis

submitted by

José Manuel Robles Aguilar

supervised by:

Dr. Damián Jacinto-Méndez

Dr. Mauricio Demetrio Carbajal-Tinoco

Mexico City, 2025

Acknowledgements

I am deeply grateful to my family for all their support throughout my academic career, especially to my parents, Fani Aguilar and Noé Robles, who have always shown me their unconditional love, and to my sisters Noemi, Noelia and Guadalupe, and my brother Noé, who always supported me both personally and academically.

Special thanks to Dr. José Miguel Méndez Alcaraz, Dr. Ricardo López Fernández and the University of Chicago for sponsoring my participation at the AI+Science Summer School in Paris. This was my first international academic experience during my master's studies and one of the best. Such experience made a lasting impact on my scientific career and helped me connect with such kind and talented people from all around the world, inspiring me to push myself to become a better scientist and person.

To my advisor Dr. Damián Jacinto Méndez, thank you for always being supportive and patient, and for trusting me with this amazing research project. I also wish to thank my committee members, Dr. Pedro González Mozuelos and Dr. José Matías Alvarado Mentado, for taking the time to review my work and provide valuable feedback.

To my colleagues and friends Ángel, Tonatiuh, Andrés, and Nadia, who made my stay at Cinvestav more enriching and enjoyable.

Finally, I would like to express my appreciation to the Secretaría de Ciencia, Humanidades, Tecnología e Innovación (SECIHTI), which funded my master's degree.

Abstract

Circular Dichroism (CD) spectroscopy is a versatile and rapid method for characterizing the secondary structure, conformation, and folding of proteins. While not a high-resolution technique, it is invaluable for analyzing protein-ligand interactions, validating computationally predicted structures, and assessing the effects of mutations. The ability to accurately predict CD spectra from structural information is crucial for these applications. The Knowledge-based Circular Dichroism (KCD) method has shown superior accuracy in predicting far-UV CD spectra by using a model with complex atomic polarizabilities. However, its predictive power can be further enhanced by refining these polarizabilities. In this work, we present a novel approach to optimize the KCD method by implementing deep Bayesian neural networks (BNNs) to refine the weights of these polarizabilities. This new, integrated model, KCD-AI, is shown to significantly improve the accuracy of far-UV CD spectral predictions by 40% when compared to the original KCD method. Furthermore, KCD-AI demonstrates superior performance against other state-of-the-art prediction tools. This refined computational model offers a powerful new tool for the structural characterization of proteins.

Contents

Acknowledgements

Abstract **i**

1 Introduction **1**

1.1 The Significance of Protein Structure 1

1.2 Protein Structure 2

1.2.1 Noncovalent Interactions 5

1.2.2 The Protein Folding Problem 7

1.3 Hierarchical Structure of Proteins 8

1.3.1 Primary Structure 8

1.3.2 Secondary Structure 8

1.3.3 Tertiary Structure 8

1.3.4 Quaternary Structure 10

1.4 Experimental Methods for Structural Characterization 10

1.4.1 X-ray Crystallography 11

1.4.2 Nuclear Magnetic Resonance (NMR) 12

1.4.3 Cryo-electron Microscopy (cryo-EM) 14

1.5 The Protein Data Bank (PDB) 16

1.6 Circular Dichroism (CD) 16

1.6.1 Secondary Structure Characterization 17

1.6.2 The Protein Circular Dichroism Data Bank (PCDDB) 20

1.6.3 Computational Prediction of CD Spectra 21

1.7 Thesis Objective 24

2 Theoretical Background **26**

2.1 The DeVoe Approach 26

2.2 The Knowledge-Based Model of Circular Dichroism (KCD) 29

2.3 Introduction to Neural Networks . . . 33
2.3.1 Biological and Artificial Neural Networks . . . 34
2.3.2 The Basics of Neural Networks . . . 35
2.3.3 The Perceptron . . . 36
2.3.4 Basic Components of Neural Architectures . . . 38
2.3.5 Multilayer Neural Networks . . . 44
2.3.6 Structured Data and Advanced Architectures . . . 46
2.4 Bayesian Neural Networks . . . 46
2.4.1 The Bayesian Framework . . . 48
2.4.2 Aplication to Neural Networks . . . 48

3 Methodology **50**
3.1 Data Collection and Preprocessing . . . 50
3.2 KCD Weight Optimization . . . 52
3.3 Data Augmentation . . . 53
3.4 Neural Network Architecture and Training . . . 57
3.5 Integration with the KCD Model . . . 62

4 Results and Discussion **64**
4.1 Neural Network Performance . . . 64
4.2 Impact on KCD Predictions . . . 67
4.3 Comparison with State-of-the-Art Methods . . . 72

5 Conclusions **75**

Bibliography **76**

Chapter 1

Introduction

In this chapter, we aim to establish the relevance of protein structure and to explain fundamental protein concepts, with a particular emphasis on properties pertinent to the work presented in this thesis.

1.1 The Significance of Protein Structure

Proteins are fundamental macromolecules, formed by stringing together 20 different amino acids in a particular sequence. This sequence dictates how the macromolecule folds, giving the protein a distinctive three-dimensional structure that is crucial for its function. Indeed, depending on their precise folded structure and genetically specified amino acid sequence, proteins can perform a wide variety of useful functions. For example, many proteins possess reactive sites that allow them to act as enzymes, accelerating chemical reactions or the replication of macromolecules such as DNA and RNA. The cytoskeleton, which supports the cell, is composed of long protein fibers, some of which have important roles, such as molecule transport. Proteins can also change their shape in response to environmental cues such as temperature, ion concentration, and pH, allowing them to act as sensitive biosensors. Other proteins bind to specific sections of DNA, thereby activating or deactivating genes.

Given the critical link between protein folding, conformation, and function, studying their structure is essential for addressing diseases that result from mutations or post-translational modifications. For example, protein misfolding and aggregation are associated with many degenerative conditions. Alzheimer's disease, for example, involves the misfolding of β-amyloid and τ proteins in the brain, and the accumulation of α-synuclein is associated with Parkinson's disease [1, 2].

In biopharmaceutical development, a thorough understanding of protein structure allows for the design and engineering of proteins with specific properties, ensuring their correct function in therapeutic applications [3]. Furthermore, this knowledge is fundamental for predicting how these proteins will behave in the organism, leading to the development of safer and more effective drugs [4].

1.2 Protein Structure

Proteins consist of unique sequences of 20 different amino acids, each linked by a covalent peptide bond [5]. A peptide bond forms between the carboxyl group of one amino acid and the amino group of the adjacent amino acid, resulting in a polypeptide chain (Figure 1.1). An amino acid within a polypeptide chain is also known as a residue. The polypeptide backbone of a protein refers to the repeating sequence of amide nitrogen (N), α-carbon (C_α), carbonyl carbon (C), and oxygen atoms. Attached to this backbone are the amino acid side chains, which are the variable groups that distinguish the 20 different amino acids and are not involved in the formation of the peptide bond (Figure 1.2). These side chains possess diverse chemical properties, including being nonpolar and hydrophobic, positively or negatively charged, or capable of forming covalent bonds.

The formation of a polypeptide chain is subject to specific physical and chemical constraints that restrict the protein's possible three-dimensional shape (or conformation). First, due to steric hindrance and the finite van der Waals radii of atoms, certain rotations that would cause atoms to clash are disallowed. Second, the peptide bond has a partial double-bond character due to electron delocalization. This property locks the six atoms of the planar peptide group - $C_\alpha - C - N - H - O - C'_\alpha$ - into a rigid plane. Third, while the peptide bond itself is rigid, rotation is permitted about the $N{-}C_\alpha$ and $C_\alpha{-}C$ bonds (Figure 1.3). These rotations are described by the dihedral angles phi (ϕ) and psi (ψ), respectively.

Despite the backbone constrictions, the extensive length of polypeptide chains still allows for a vast number of possible three-dimensional conformations. Indeed, in the 1960s, Cyrus Levinthal highlighted that there is a huge number of possible con-

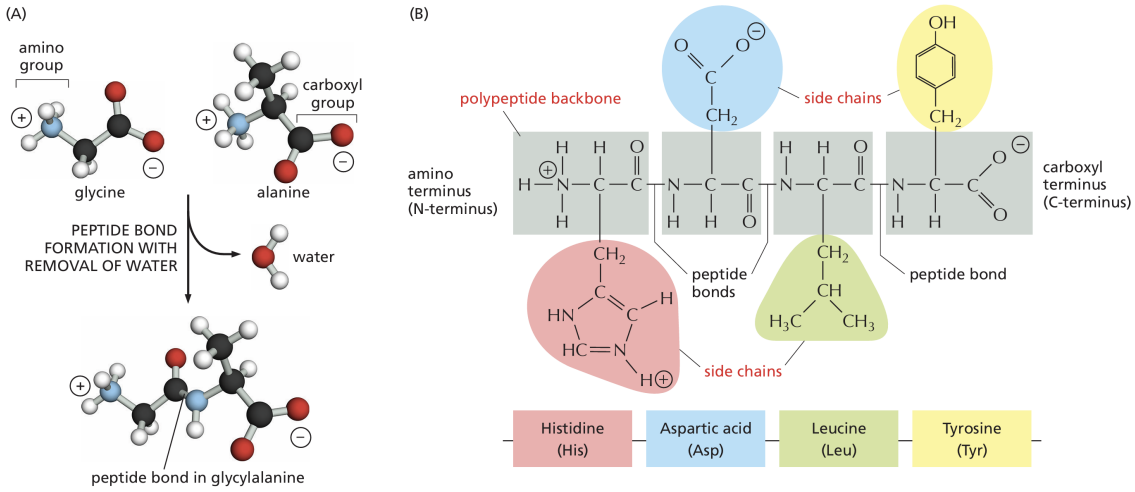


Figure 1.1: **(A)** The peptide bond is a covalent bond formed between the carbon atom of the carboxyl group of one amino acid and the nitrogen atom of the amino group of a second amino acid. This reaction is a condensation reaction, or dehydration synthesis, because a water molecule is eliminated in the process. Atoms are represented by standard colors: black for carbon, blue for nitrogen, red for oxygen, and white for hydrogen. **(B)** A short section of the polypeptide backbone is shown with its attached side chains, along with a representation of the corresponding amino acid sequence. The polypeptide chain is always read and represented from the N-terminus to the C-terminus (left to right), which corresponds to the direction of polypeptide synthesis in the cell [5].

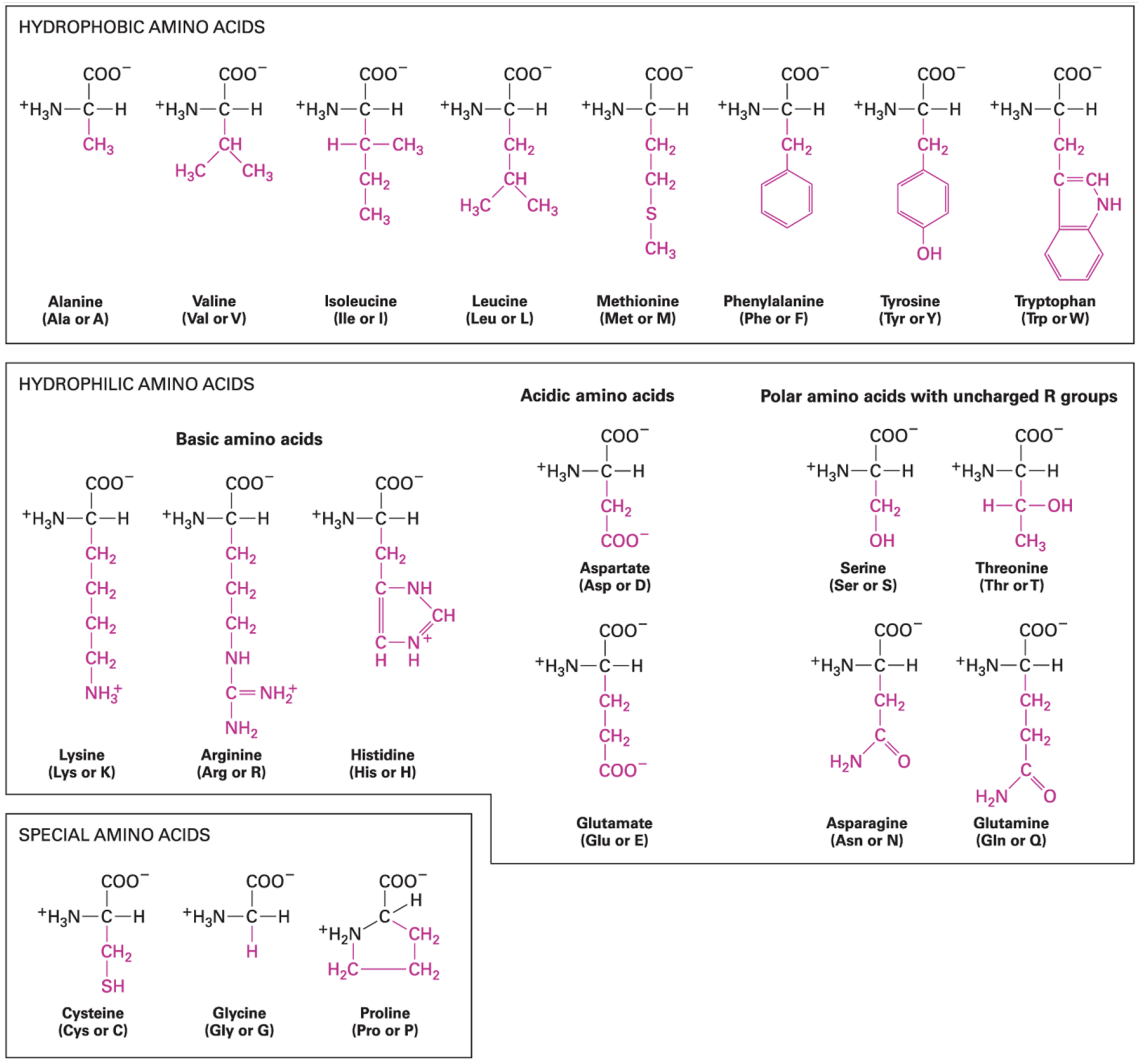


Figure 1.2: The 20 canonical amino acids found in proteins and their unique properties. The side chains (R-groups), shown in pink, are the key functional components that define the distinct chemical characteristics of each amino acid. The three- and one-letter abbreviations for each amino acid are listed in parentheses [6].

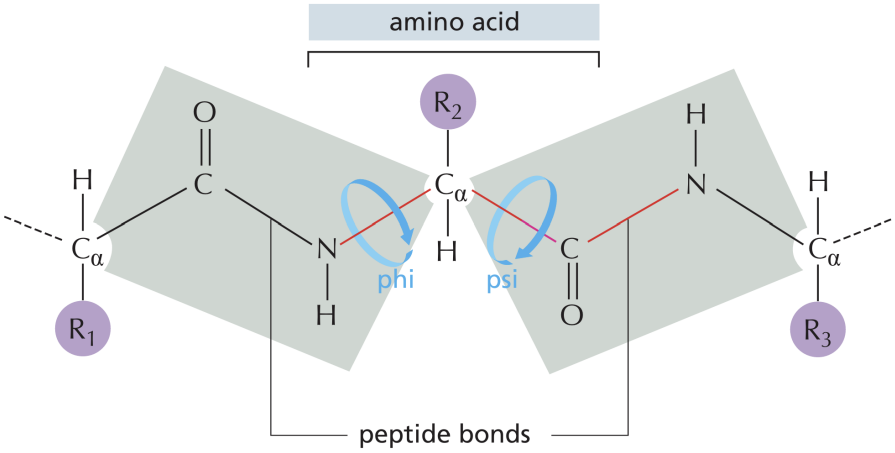


Figure 1.3: The key rotational features of the polypeptide backbone. The peptide bond itself, shown with gray shading, is planar and rigid, preventing free rotation. Rotation is only possible around two single bonds for each amino acid residue: the phi (ϕ) angle, which is the rotation around the N$-$C$_\alpha$ bond, and the psi (ψ) angle, which is the rotation around the C$_\alpha-$C bond. The side chains of the amino acids are shown in purple circles, and the two rotating bonds are highlighted in red [5].

formations a protein chain could theoretically adopt. If a protein were to randomly sample all these possible configurations, it would take an incomprehensible amount of time. For instance, suppose a protein is 100 amino acids long and each amino acid has 3 possible backbone conformations. Then the total possible conformations are 3^{100}, which is approximately 5×10^{47}. Now, suppose the protein samples one conformation per nanosecond. The total time to explore all conformations would then be 5×10^{47} conformations $\times\, 10^{-9}$ seconds/conformation $= 5 \times 10^{38}$ seconds. This astronomical time, approximately 10^{31} years, is vastly longer than the age of the universe ($\sim 10^{10}$ years). Given that proteins, in reality, fold rapidly within milliseconds to seconds, how can they find their specific native structure so quickly if the search space is immensely large? This fundamental question is famously known as Levinthal's Paradox [7].

1.2.1 Noncovalent Interactions

The stabilization of a protein's unique shape is based primarily on four types of noncovalent interactions. Individually, these interactions are significantly weaker than

covalent bonds (typically 30-300 times), but their collective cumulative strength, arising from their large number within a protein, is crucial for maintaining structural integrity.

Electrostatic Attractions (Ionic Bonds)

These interactions arise from the electrostatic attraction between a cation (a positively charged ion) and an anion (a negatively charged ion). A key characteristic differentiating them from covalent bonds is their lack of specific directional geometry, which stems from the uniform nature of the electrostatic field surrounding the ions.

Hydrogen Bonds

Hydrogen bonds are a crucial type of dipolar electrostatic interaction, formed between a partially positively charged hydrogen atom (covalently bonded to an electronegative atom, such as oxygen or nitrogen) and a lone pair of electrons on another electronegative atom. This interaction can occur within the same molecule (intramolecular) or between different molecules (intermolecular), as exemplified by water. Hydrogen bonds exhibit distinct directional preferences: the strongest bonds occur when the donor (the electronegative atom covalently bonded to hydrogen), the hydrogen atom, and the acceptor (the electronegative atom with a lone pair) are aligned in a straight line. Although nonlinear hydrogen bonds are weaker, they nonetheless contribute significantly to the overall stability of many protein structures.

Van Der Waals Attractions

The van der Waals attractions come into play when any two atoms approach each other closely. These weak forces arise from transient fluctuations in the distribution of electrons within every atom. Such random fluctuations lead to a temporary, unequal distribution of electrons, thereby forming a temporary dipole. A neighboring molecule is then influenced by this temporary dipole, which in turn perturbs its own electron cloud and induces a secondary, temporary dipole. The resulting weak attraction between these two transient dipoles constitutes a van der Waals interaction. Critically, these interactions occur in both polar and nonpolar molecules.

Hydrophobic Interaction

Another important interaction that significantly influences the three-dimensional structure of a protein is the hydrophobic effect. This phenomenon arises from the interaction between nonpolar molecules and water. Due to their inability to form hydrogen bonds with nonpolar substances, water molecules tend to form highly ordered, hydrogen-bonded pentagon and hexagon cages around nonpolar molecules. This ordered arrangement represents a state of low entropy for the water. In an aqueous environment, hydrophobic molecules, such as the nonpolar side chains of certain amino acids in a protein, will thus tend to aggregate or cluster together. This aggregate reduces the total surface area exposed to water, thereby releasing a large number of ordered water molecules into the bulk solvent, which significantly increases the entropy of the system and drives the overall process towards a state of lower free energy. In contrast, polar groups in the side chains will tend to arrange near the molecule's surface, where they can form favorable hydrogen bonds with water, other polar amino acids, or the polypeptide backbone.

1.2.2 The Protein Folding Problem

The noncovalent interactions collectively establish a protein's characteristic three-dimensional structure. Under standard physiological conditions, and in accordance with Anfinsen's Dogma [8], this structure is uniquely determined by its amino acid sequence. It corresponds to the global minimum of the system's Gibbs free energy. This principle inherently addresses Levinthal's Paradox, as it implies a directed, rather than random, folding pathway. Consequently, a central challenge in biochemistry is to predict the final folded structure of a protein solely from its amino acid sequence. This fundamental challenge is known as the protein folding problem.

The protein folding problem has been tackled by diverse computational and experimental approaches, where artificial intelligence has played an increasingly significant role. A prominent example is AlphaFold2, which has achieved unprecedented experimental accuracy in protein structure prediction [9].

1.3 Hierarchical Structure of Proteins

The structure of proteins is hierarchical, organized into distinct levels that represent increasing complexity. These levels of organization are designated as the primary, secondary, tertiary, and quaternary structures (Figure 1.4).

1.3.1 Primary Structure

As previously discussed, amino acids, commonly termed residues, are the fundamental units that define a protein's identity and properties. The primary structure of a protein is defined as the specific linear sequence of these residues within the covalently linked polypeptide chain. This sequence is conventionally represented from the N-terminal end (on the left) to the C-terminal end (on the right), with each residue assigned a sequential position beginning at the N-terminus.

1.3.2 Secondary Structure

The secondary structure of a protein is defined as the stable, repeating structural patterns formed by hydrogen bonds between the atoms of the polypeptide backbone. Specifically, these hydrogen bonds form between the carbonyl oxygen of one residue and the amide hydrogen of another, creating stable segments. The most common secondary structures are the alpha (α) helix, the beta (β) sheet, and the beta (β) turn. The formation of a particular secondary structure is highly dependent on the amino acid sequence of that segment. Other regions that are highly flexible and lack a defined three-dimensional structure are described as random coils or intrinsically disordered regions.

1.3.3 Tertiary Structure

The tertiary structure corresponds to the overall three-dimensional arrangement of a single polypeptide chain. This unique fold is stabilized by a combination of noncovalent and covalent interactions, with the chemical properties of amino acid side chains playing a

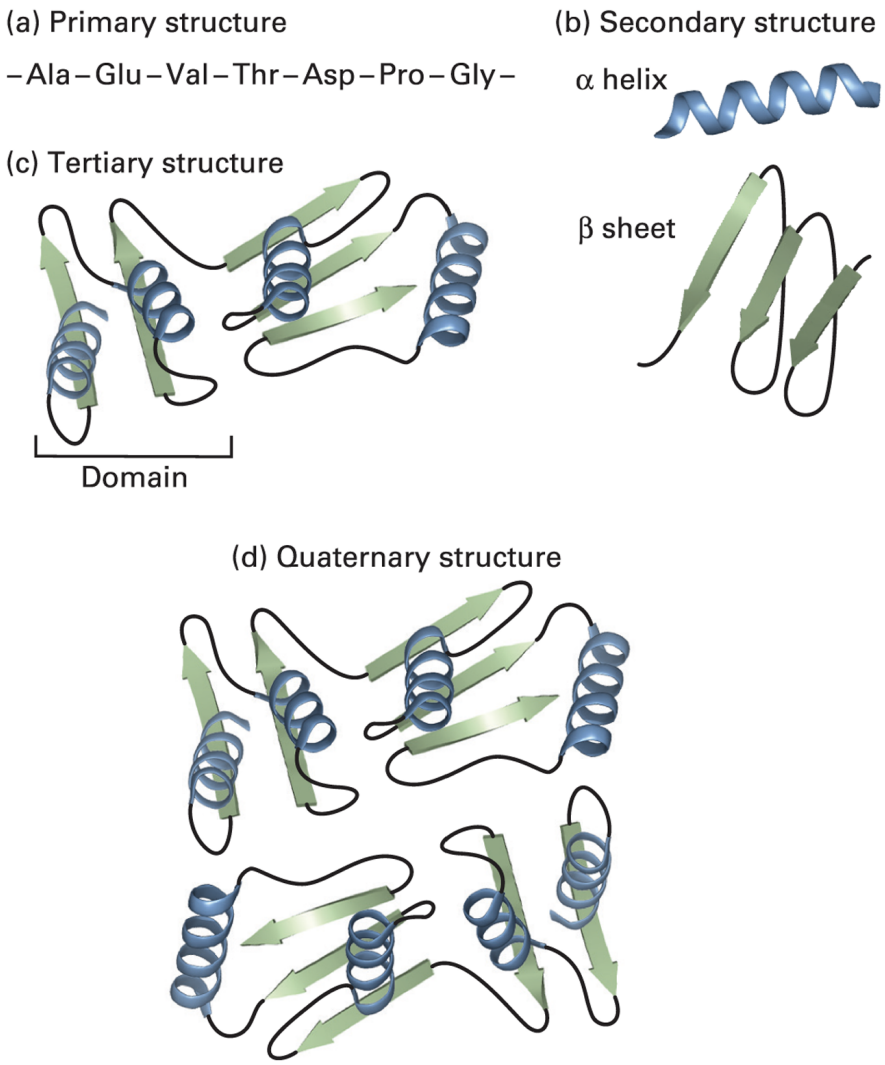


Figure 1.4: The hierarchical organization of protein structure. (a) Primary structure is the linear sequence of amino acids joined together by peptide bonds. (b) Secondary structure refers to the local folding of the polypeptide chain into stable, repeating patterns, such as α-helices and β-sheets. (c) Tertiary structure represents the overall three-dimensional arrangement of a single polypeptide chain, which is the result of the spatial organization of its secondary structural elements, loops, and turns. (d) Quaternary structure is the result of the non-covalent association of multiple folded polypeptide chains, forming a functional protein complex [6].

key role. The main stabilizing forces include hydrophobic interactions between nonpolar side chains, as well as hydrogen bonds and van der Waals attractions. Crucially, ionic bonds between oppositely charged side chains also provide significant stability, as do disulfide bonds formed between two cysteine residues. The tertiary structure organizes various elements of secondary structure, and in contrast to secondary structure's local rigidity, it is not necessarily fixed. Instead, it often undergoes continual thermal fluctuations that are essential for protein function.

1.3.4 Quaternary Structure

Multimeric proteins are composed of two or more polypeptide chains, referred to as subunits. These subunits can be identical (homomeric) or different (heteromeric) and are held together by noncovalent interactions, as well as sometimes by covalent disulfide bonds. The quaternary structure describes the number (stoichiometry) and spatial arrangement of these subunits in the final protein complex.

1.4 Experimental Methods for Structural Characterization

Since we cannot directly visualize a protein's structure, we must rely on methods that measure its physical properties. The images and models presented in the preceding section are representations of a protein's atomic positions in three-dimensional space, and their reliability is contingent upon a strong agreement with these measured properties. Methods for characterizing protein structure typically involve interactions with light, magnetic, or electric fields, or movement against a free energy gradient. This section will focus on three of the most well-established methods: X-ray crystallography, nuclear magnetic resonance (NMR) spectroscopy, and cryo-electron microscopy (cryo-EM).

1.4.1 X-ray Crystallography

The initial technique developed to study the protein structure at the atomic scale was X-ray crystallography. The groundbreaking work of John Kendrew and Max Perutz yielded the first protein crystal structure, myoglobin, in 1958 [10]. Since then, the methodology has evolved significantly, with key advancements in crystallization techniques and new computational approaches for data processing and model building. The main steps for determining a protein structure using this method are as follows [11]:

Sample Preparation

The protein must first be purified using techniques such as chromatography before being concentrated to a level suitable for crystallization. Crystals are then grown under carefully controlled conditions of temperature, pH, and concentration, facilitating the organization of protein molecules into the highly structured crystalline state that can be subjected to X-ray diffraction.

X-ray Diffraction

A narrow beam of parallel X-rays is directed at the crystal. A small fraction of the X-rays is scattered by the electron clouds of the atoms in the crystal. Because the X-ray wavelength (around 0.1 nm) is similar to the distances between atoms, these scattered waves constructively interfere at specific points, producing a diffraction pattern. This pattern is recorded by a suitable detector. To ensure comprehensive data collection, the crystal is rotated 360° during data acquisition.

Data Analysis

Information about the atomic locations in the crystal is inferred from the position and intensity of each spot in the diffraction pattern. The pattern is analyzed using computer-assisted mathematical algorithms based on Fourier transforms, which generate a three-dimensional electron density map.

Model Building

Following a complex procedure, the known amino acid sequence is correlated with the electron density map to achieve the best fit, thus generating the initial atomic structure of the protein.

Refinement

The protein structure model is refined iteratively to better align with the experimental data. At high resolution (typically less than 1.5 Å), it is feasible to generate a model with correct bond angles and lengths based on the electron-density map. At lower resolutions, specialized visualization software is necessary to manually match the model to the electron density data.

Validation

The quality of the final protein structure is rigorously assessed. This process entails inspecting the electron density map to resolve steric overlaps and verifying that geometric constraints, such as bond lengths and angles, are physically reasonable. Furthermore, the model is cross-referenced with the established biochemical and biophysical characteristics of the protein to ensure consistency.

1.4.2 Nuclear Magnetic Resonance (NMR)

NMR spectroscopy has a long history of use in the analysis of the structure of small molecules, small proteins, and protein domains. Unlike X-ray crystallography, there is no need for crystalline samples; instead, a concentrated protein solution is sufficient for structural characterization. This makes NMR an invaluable tool for proteins that are difficult to crystallize or whose crystals are not of sufficient quality for high resolution.

The fundamental basis of NMR spectroscopy is the determination of the Larmor frequency, defined as the rate at which a nuclear magnetic moment precesses when subjected to a strong external magnetic field. Because this frequency is modulated by the specific chemical environment of each nucleus, it provides detailed information

about the structure of the protein. Beyond static structure determination, NMR is also widely used to characterize a protein's conformational changes, internal dynamics, and interactions at atomic resolution.

The steps to characterize protein structure by NMR spectroscopy are as follows [11]:

Sample Preparation

The protein to be analyzed must be dissolved in a suitable solvent buffer at a concentration of approximately 0.1 to 4 mM. Solvent buffers are essential for maintaining protein stability, ensuring the molecule remains folded, soluble, and functional throughout the NMR experiment. To enhance spectral resolution and facilitate resonance assignment, Isotopic labeling can be employed, a technique in which naturally abundant, NMR-silent isotopes are replaced with NMR-active ones (e.g., ^{13}C or ^{15}N). It is critical to prevent protein aggregation or degradation during this process to preserve the protein's native state.

NMR Experiment

When the sample is put in a powerful magnetic field, the magnetic moments of some atomic nuclei (due to their intrinsic spin) align with the field. This alignment is perturbed by radiofrequency pulses of electromagnetic radiation. The resulting NMR signals, or responses, are measured and detected as a spectrum. The specific frequency of each nucleus depends on its chemical environment, therefore providing structural information.

Data Collection

The measurement and detection of NMR signals are performed using various pulse sequences. The frequencies and intensities are recorded to generate a multidimensional spectrum, which contains information about the positions and distances of atoms within the protein.

Spectral Analysis

From the NMR spectra, key parameters such as chemical shifts, coupling constants, and relaxation rates are inferred. These parameters provide critical insights into the structure, dynamics, and interactions of the protein.

Structure Calculation

The final protein structure is determined by computationally simulating the protein's fold using minimization methodologies under experimental constraints. These constraints predominantly consist of inter-atomic distance limits obtained from Nuclear Overhauser Effect Spectroscopy (NOESY) experiments and dihedral angles (ϕ and ψ), which are derived from the ^{1}H, ^{15}N, and ^{13}C backbone chemical shifts and scalar coupling constants. To produce a model that adheres to these experimental boundaries, computational techniques such as distance geometry, simulated annealing, or Molecular Dynamics (MD) simulations are employed.

Validation

The quality of the final NMR models is assessed using specialized tools such as PROCHECK-NMR, MolProbity, PSVS, or CING. These tools evaluate stereochemical correctness and overall structural quality to confirm the model's accuracy.

1.4.3 Cryo-electron Microscopy (cryo-EM)

One of the main challenges in structural biology is the characterization of large protein complexes, as these are often difficult to crystallize (a limitation for X-ray crystallography) and too large for NMR spectroscopy. Cryogenic electron microscopy (Cryo-EM), a technique that began development in the 1980s [12], has revolutionized the field by enabling the study of large proteins and supramolecular structures in their native states without the need for crystallization. This has made it possible to determine the structures of complexes, including viral particles, amyloid fibers, and protein-protein assemblies.

The Cryo-EM technique involves rapidly freezing the aqueous specimen to form vitreous ice and capturing electron microscopy images. These images are then used to precisely reconstruct the protein's three-dimensional structure. The general steps to determine a protein structure using Cryo-EM are as follows [11]:

Sample Preparation

Approximately 3-5 μL of the protein solution is applied to a carbon grid. The excess liquid is then blotted away, and the sample is flash-frozen to form a thin layer of vitreous ice. Special care is taken to prevent protein aggregation or absorption onto the grid surface, which could compromise the final structure.

Data Acquisition

Using an electron microscope under cryogenic conditions, multiple two-dimensional (2D) images, or electron micrographs, of the frozen sample are recorded. For single-particle analysis, the images are taken without tilting, relying on the random orientation of particles to capture different views. For electron tomography, images are recorded at different tilt angles to build a comprehensive dataset.

Image Processing

The acquired 2D images are processed using techniques such as single-particle analysis or electron tomography. This process involves correcting for imperfections in the microscope and computationally aligning the particles to reconstruct a three-dimensional (3D) density map of the protein.

Model Building

The reconstructed 3D density map is fitted with an atomic model, either manually or with computational tools. The coordinates of the atoms are positioned and refined to achieve maximum agreement with the experimental density data, resulting in a structural model of the protein.

Validation

The final model's quality is rigorously assessed. This involves checking that the atomic model fits the density map without significant steric clashes and evaluates the model's geometric parameters. Validation is often confirmed through a combination of methods, including the Fourier Shell Correlation (FSC) to determine the map's resolution, cross-validation against independent datasets, and comparison with known biochemical or biophysical data related to the protein's function.

1.5 The Protein Data Bank (PDB)

The rapid development of experimental techniques for structural characterization of proteins led to the obtainment of an increasing number of determined structures, necessitating the creation of a centralized database. The Protein Data Bank (PDB) [13] was founded as an initiative to archive biological macromolecular crystal structures, including their atomic coordinates, structure factors, and electron density maps. It was created in 1971 at the Brookhaven National Laboratories (BNL) and the Cambridge Crystallographic Data Centre, initially under the leadership of Walter Hamilton with just seven structures [14, 15]. For over 50 years, this database has remained an invaluable tool for structural bioinformatics, standardizing protein structure representation and creating quality control methodologies. As of August 2025, the PDB contains more than 240,000 protein structures [16].

1.6 Circular Dichroism (CD)

An important property of the amino acids that compose proteins is chirality. A molecule is chiral when it is not superimposable on its mirror image. Chirality in amino acids arises from the presence of an asymmetric carbon, a carbon atom tetrahedrally bonded to four different atoms or groups. Glycine, which has a hydrogen atom as its side chain, is the only achiral amino acid.

Chiral structures can be distinguished and characterized by Circular Dichroism (CD)

spectroscopy, a technique based on measuring the differential absorption of left- and right-circularly polarized light. For proteins, CD occurs in the near and far ultraviolet (UV) wavelength ranges, where the amide and carbonyl groups of the polypeptide backbone absorb light.

Although Circular Dichroism (CD) spectroscopy is not a high-resolution technique comparable to X-ray crystallography, NMR spectroscopy, or Cryo-EM, from which structure determination can take hours, months, or even years depending on the specific protein and method, it remains a principal tool for studying protein secondary structure [17], conformation and function [18], protein-protein interactions [19], and protein aggregates [20]. Its key advantages over these other methods include requiring a relatively small amount of sample, allowing for experiments under near-native conditions, and providing rapid results [21].

1.6.1 Secondary Structure Characterization

The usual steps for protein secondary structure characterization using CD spectroscopy are as follows:

Sample Preparation

To ensure high-quality data, the protein of interest must have a purity of at least 95%. Standard protein purification methods such as high-performance chromatography, mass spectrometry, or gel electrophoresis are recommended to achieve this purity. The protein should be dissolved in a buffer that is as transparent as possible and free of any optically active materials that could interfere with the CD signal.

CD spectra are typically collected using high-transparency quartz cuvettes, or cells, which are either rectangular or cylindrical with pathlengths ranging from 0.01 to 1 cm. The specific pathlength determines the appropriate sample concentration, which can range from 0.005-5 $mg\ ml^{-1}$. Quantitative amino acid analysis is considered the most accurate method for determining protein concentration, as it calculates the concentration of the intact protein from the amounts of its stable amino acid components. Finally,

the total absorbance of the sample, including the buffer and cell, must remain below 1 throughout the wavelength scan to ensure accurate, high-quality data [17].

CD Spectra Collection

For accurate determination of protein secondary structure, measurements must be conducted over a wavelength range that includes at least 190 to 240 nm. Extending the scan to include wavelengths below 190 nm will increase the information content of the data, leading to more precise results.

Commercial CD instruments measure the ratio of two currents at the detector. The first is an AC current, which corresponds to the difference in photon intensities of left- and right-circularly polarized light (I_L and I_R) that is transmitted through the sample. The second is a DC current, which represents the average of the two intensities,

$$AC/DC = \frac{2(I_L - I_R)}{(I_L + I_R)}. \tag{1.1}$$

This measurement is used to determine the differential absorbance, or CD signal, $\Delta A = A_L - A_R$, and can be expressed by the following relationship:

$$\Delta A = log_{10}\left(\frac{I_R}{I_L}\right). \tag{1.2}$$

The dynode voltage, also known as the high-tension (HT) signal, is a measure of the voltage applied to the detector to amplify the small CD signal. It is also used to monitor the overall absorbance of the sample. In the most common operating mode, the HT voltage is adjusted to keep the DC current constant. The software of the instrument then approximates the differential absorbance by the following formula:

$$\Delta A = log_{10}\left(\frac{2 + AC/DC}{2 - AC/DC}\right) \approx \frac{AC}{DC} log_{10}(e). \tag{1.3}$$

The approximation error in this equation is less than 1% if the CD signal is less than 20% of the absorbance [22].

To ensure high data quality, repeated CD spectra of both the sample and a baseline (buffer only) are collected. During this process, the HT signal is monitored to confirm it

remains within the instrument's linear range. After collection, the data are averaged, and any outliers are removed. The averaged baseline spectrum is then subtracted from the averaged sample spectrum, with the final spectra being aligned at wavelengths greater than 250 nm, where no protein signal is expected. With the concentration, optical cell pathlength, and mean residue weight value for the protein, the final spectrum is normalized to standard units of mean residue ellipticity (MRE, degrees cm^2 $dmol^{-1}$ $residue^{-1}$) or differential molar absorption coefficient ($\Delta\varepsilon$, M^{-1} cm^{-1})[23].

Data Analysis

CD spectra analysis is grounded in the idea that a protein's overall spectrum is essentially a sum of its individual secondary structure components, plus some interference from things like aromatic residues or prosthetic groups [17]. To figure out the secondary structure from an experimental spectrum, researchers use a reference set of proteins with already solved structures as a baseline for comparison. Mathematical tools are then used to match the calculated data to the observed spectrum. These tools vary in complexity, starting with standard least squares regression and moving to more advanced methods like ridge regression, singular value deconvolution (often with variable selection), and even neural networks trained on the reference data [23].

The most common analysis programs include the DichroWeb, BeStSel, and K2D3 servers. These platforms offer a range of methods and reference datasets, all of which yield similar results for proteins with high alpha-helix content. The BeStSel method, however, is particularly noted for its strength in characterizing beta sheet-rich proteins. A limitation of analysis methods that rely on a reference to known protein structures is their difficulty in accurately characterizing intrinsically disordered proteins (IDPs), as these proteins tend not to crystallize. Consequently, there are very few structures of IDPs in the Protein Data Bank (PDB) that are not composed of the main secondary structure types [23].

Validation

The ValiDichro platform serves as a validation tool for Circular Dichroism (CD) spectroscopy, designed to detect spectral artifacts, data inconsistencies, and errors in the parameters used for secondary structure calculations. The software evaluates experimental data by benchmarking it against established spectral characteristics derived from the literature. Based on this comparison, the system assigns a quality score: a 'Pass' (P) for compliant data, a 'Flag' (F) for minor deviations, or a 'Fail' (X) for significant errors [23].

The validation process includes a comprehensive suite of tests, such as verifying that peak magnitudes (measured in $\Delta\varepsilon$ or MRE) fall within expected biological ranges. The software also scans for outliers that may indicate inaccuracies in protein concentration or optical pathlength, as well as sample-specific issues like peak flattening caused by light scattering or excessive absorbance. Instrumental performance is assessed by ensuring the High Tension (HT) signal does not exceed the detector's limit and by confirming that the gradient in the non-absorbing 240–260 nm region remains flat. Additionally, peak positions are analyzed for shifts that might suggest calibration or absorbance issues. The final output provides a status indicator for each test along with actionable recommendations for data correction [23].

1.6.2 The Protein Circular Dichroism Data Bank (PCDDB)

The Protein Circular Dichroism Data Bank (PCDDB) [24] is a freely accessible repository of validated CD and synchrotron radiation circular dichroism (SRCD) spectra and associated metadata. Released in 2010, the PCDDB was inspired by resources such as the Protein Data Bank (PDB), the UniProt sequence database, and the Enzyme Classification IntEnz (EC) database. Initially, it served as an accession-only database for CD reference spectra.

Today, the PCDDB allows user deposition, enabling authors and publishers on CD topics to make their data and metadata publicly available. As of August 2025, the PCDDB contains 741 deposited spectra [25]. It also hosts programs for data processing,

analysis, and display of CD spectroscopic data. While access to and download of data is freely available without registration, author registration is required for data deposition to ensure good practice and traceability of the data [26].

1.6.3 Computational Prediction of CD Spectra

Computational prediction of CD spectra is particularly useful for comparing two proteins when the high-resolution structure of one protein (determined by methods such as X-ray diffraction, cryo-EM, or NMR) or a predicted structure (e.g., from AlphaFold or similar tools) is available, but only the CD spectrum of the second protein is known. These comparisons serve various purposes, including validating structural similarity (homology), assessing the folding of mutated proteins, observing the effects of ligand binding, or environmental factors on protein conformation, and determining whether modeled (or predicted) structures have CD spectra similar to experimental spectra. Common methods for CD spectra prediction include the following:

The DichroCalc Server

The DichroCalc server is a web interface that predicts CD spectra using a matrix method derived from first-principles calculations. These calculations consider the far-UV peptide chromophore, charge-transfer between neighboring peptide groups in the deep-UV, and aromatic side chain chromophores with transitions in the near-UV. The server accepts Protein Data Bank (PDB) files as input, either as a single file or as a compressed archive, and also supports PDB codes separated by commas. The final calculation is delivered as a two-column text file containing wavelength and corresponding CD values [27, 28].

The PDB2CD Server

The PDB2CD server is an empirical approach for predicting CD (or SRCD) spectra from protein three-dimensional atomic coordinates in PDB file format. The input can be a structure determined by experimental methods, such as X-ray crystallography or

NMR (in which the first model structure listed will be used), or it can be a theoretical structure from molecular dynamics simulations or other modeling tools. PDB2CD utilizes either the SP175 or the SMP180 spectral datasets from the PCDDB, with the user having the option to select between them. To generate a spectrum from a given protein structure, PDB2CD employs three complementary structure-based approaches that operate at different levels of detail. The first approach is based on the percentage content of secondary structures (alpha-helix, beta-strand, or others) as determined by the Dictionary of Secondary Structure of Proteins (DSSP) values calculated from the input PDB file. The second method compares topological features of neighboring secondary structure components, which are defined by helix axes and strand vectors. The final approach assesses the overall structural similarity between the query protein and a subset of the proteins in the reference dataset. By weighting the data of similar proteins within each of these three methods, the server minimizes the difference between the generated and experimental spectra. The final output is a plot showing the predicted spectrum superimposed on the experimental spectrum [29].

The PDBMD2CD server

The PDBMD2CD server is a successor to PDB2CD, offering a significantly faster and more accurate approach to CD spectra prediction. It can predict CD spectra from multiple protein structure files, such as those derived from molecular dynamics (MD) simulations, generating a spectrum for each structure provided by the user. It also incorporates new analysis tools for clustering predictions or comparing them against experimental CD spectra.

PDBMD2CD uses two primary approaches. The first involves using basis spectra derived from a least-squares regression of the CD signal from structures in a reference set based on the SMP180 dataset. The second approach creates a basis set of spectra from reference structures that have the closest secondary structure content to the target protein, followed by a multivariate optimization of the contributing spectra's summed secondary structure content in the final prediction. The final output is an average of these two approaches to achieve higher accuracy. The server accepts PDB structure files

in either PDB or Macromolecular Crystallographic Information File (mmCIF) format. Both single or multiple files can be uploaded as a compressed file, and it is also possible to use a PDB code or multiple codes separated by commas. The final output consists of three distinct pages: 'Results', 'Clustering', and 'Compare to Experiment', each accessible through a tabbed menu and providing specific types of analysis for the multiple predicted spectra [30].

The SESCA software

The structure-based empirical spectrum calculation approach (SESCA) combines elements of both deconvolution methods and spectrum prediction. Like deconvolution methods, SESCA uses secondary structure-related basis spectra. However, unlike deconvolution methods, it obtains the required coefficients from a model structure, similar to other CD prediction methods. This ensures that predictions are not affected by potential experimental errors from a measured CD spectrum of the target protein. SESCA offers a choice of different structural assignments to calculate a set of between three and eight basis spectra. A weighted average of the basis spectra derived from the secondary structural information of the target protein is then used to generate the final predicted spectrum. The use of pre-calculated basis spectra allows for fast and accurate spectrum predictions. The SESCA software requires downloading and installation in a Python environment [31, 23].

The KCD server

The Knowledge-based Circular Dichroism (KCD) server [32] utilizes a model based on the classical theory of optical activity with a complex set of atomic polarizabilities. These polarizabilities are obtained from a base of SRCD spectra and PDB structures from the PCDDB to predict far-UV CD spectra from the three-dimensional structural information of a target protein. This information includes the secondary structure content provided by the Structural Identification (STRIDE) algorithm and the atoms belonging to D-amino acid residues. According to a metric of closeness based on normalized absolute deviations (NAD) between experimental and predicted spectra, the KCD method offers

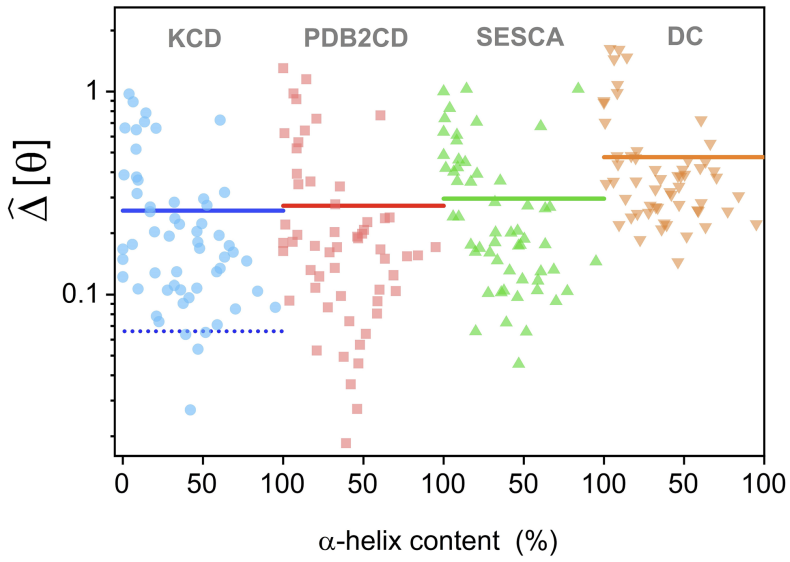


Figure 1.5: Performance of KCD against main competitors. The normalized absolute deviation ($\hat{\Delta}[\theta]$) of their Circular Dichroism (CD) predictions for 57 test proteins is shown. The results for each approach are represented by distinct symbols: KCD (blue circles), PDBMD2CD (red squares), SESCA (green triangles), and DichroCalc (orange triangles). The continuous lines show the average deviation for each approach, denoted as $\langle\hat{\Delta}[\theta]\rangle$. The dotted line indicates the average value of the KCD learning set [33].

the most accurate predictions in comparison to the DichroCalc, PDBMD2CD, and SESCA approaches (Figure 1.5). The KCD server accepts a PDB file of the target protein as input and an optional spectrum file for normalization and comparison, which should be a simple two-column ASCII file (.dat or similar). The user is also required to provide their name and email address. The results are sent to the user via email and include a plot of the experimental and predicted spectra superimposed, a bar graph with the percentage content of secondary structures (alpha, beta, coils, and others) and D-residues, and the ASCII data file of the predicted spectrum [33].

1.7 Thesis Objective

Despite the demonstrated accuracy of the Knowledge-based Circular Dichroism (KCD) method compared to other established CD prediction approaches, a significant opportunity exists to further enhance its predictive capabilities by refining the atomic

complex polarizabilities through a data-driven approach. The objective of this thesis is to develop and implement deep neural networks that optimize the weights of the complex polarizabilities in the KCD method, thereby enhancing the accuracy of far-UV CD spectral predictions.

Chapter 2

Theoretical Background

The theoretical modeling of a protein's CD spectrum has been approached through various methods. One such approach is ab initio quantum mechanics, used in programs like DichroCalc. This method involves diagonalizing a Hamiltonian matrix to generate eigenvalues and eigenvectors that correspond to the energies and electronic states of the excited states of the polypeptide. From these, the electric and magnetic dipole transition moments are calculated, which in turn yield the dipole and rotational strengths. While this approach is grounded in first principles, it is computationally intensive and often requires the introduction of several approximations [21, 27]. In contrast, other approaches are based on the statistical analysis of experimental CD spectra, as seen in the PDB2CD and SESCA prediction servers (see section 1.6.3). These methods are generally more accurate and computationally efficient than first-principles calculations. A third route, which will be expanded upon in the following section, is a classical approach that offers a unique and powerful way to model optical activity.

2.1 The DeVoe Approach

Howard DeVoe, in two seminal papers, presented a classical theoretical description of optical properties, including ellipticity, optical rotation, and extinction coefficients. This model has been widely applied to structural problems in organic, inorganic, and polymer chemistry. One of its primary advantages is its simplified framework, which treats the protein's chromophores as damped oscillators modeled as point dipoles. This provides a suitable method for deriving optical properties while allowing for the polarizabilities to be determined externally.

DeVoe's model assumes that the atoms of the chromophores composing a protein

are polarized by external electromagnetic radiation and are coupled to one another by their own oscillating dipolar fields. Each atom is represented as a point dipole, labeled i or j, at its corresponding atomic position within the protein. The dipoles are defined by an atomic polarizability tensor, $\hat{\alpha}$, which is a diagonal matrix with three components. Assuming these components can be approximated by an average complex polarizability, α_i, the polarization of each dipole, $\mathbf{m}_i = m_i \mathbf{e}_i$, under external radiation depends on the local electric field. This local field is the sum of the incident field, $\mathbf{E}$, and the fields generated by the polarization of all other chromophores [34]:

$$m_i = \alpha_i \left[\mathbf{e}_i \cdot \mathbf{E} + \sum_{j \neq i} G_{ij} m_j \right] \tag{2.1}$$

where G_{ij} is the interaction coefficient given by:

$$G_{ij} = \frac{1}{r_{ij}^3}[(\mathbf{e}_i \cdot \mathbf{e}_j) - 3(\mathbf{e}_i \cdot \mathbf{e}_{ij})(\mathbf{e}_j \cdot \mathbf{e}_{ij})], (i \neq j) \tag{2.2}$$

Here, $r_{ij} = |r_j - r_i|$ and $\mathbf{e}_{ij}$ is the unit vector of $\mathbf{r}_{ij}$. This interaction coefficient represents the contribution from a unit point dipole along $\mathbf{e}_j$ at one chromophore to the local field component along $\mathbf{e}_i$ at another chromophore, and it is closely related to the dipole interaction tensor $\hat{T}_{ij} = r_{ij}^{-3}[\mathbf{1} - 3\mathbf{e}_{ij}\mathbf{e}_{ij}]$, since $G_{ij} = \mathbf{e}_i \cdot \hat{T}_{ij} \cdot \mathbf{e}_j$.

By solving this system of linear equations, which relates the induced moment on atom i to the induced moments on all other atoms, one can obtain the induced moment on each group as a function of the external field values at the dipole location [35]:

$$m_i = \sum_j A_{ij}(\mathbf{e}_j \cdot \mathbf{E}). \tag{2.3}$$

Here, the matrix $\hat{\mathbf{A}}$ is the inverse of matrix $\hat{\mathbf{B}}$, where $\hat{\mathbf{B}}$ is defined as:

$$\hat{\mathbf{B}} = \hat{\mathbf{A}}^{-1} = \hat{\mathbf{\Lambda}}^{-1} + \hat{\mathbf{G}} \tag{2.4}$$

where $\Lambda_{ij} = \delta_{ij}\alpha_i$.

By calculating the electric polarization and the complex refractive indexes for left- and right-circularly polarized light, DeVoe's theory provides expressions for both the molar ellipticity, $[\theta]$, and the molar rotation, $[\phi]$ [36]:

$$[\theta] = \sum_{i,j} C_{ij} Im(A_{ij}) \tag{2.5}$$

$$[\phi] = -\sum_{i,j} C_{ij} Re(A_{ij}) \tag{2.6}$$

where C_{ij} is a coefficient defined as [37]:

$$C_{ij} = \frac{24\pi^2 N_A}{\lambda^2}[(\mathbf{e}_i \times \mathbf{e}_j) \cdot (\mathbf{r}_i - \mathbf{r}_j)] \tag{2.7}$$

with N_A being Avogadro's number and λ the wavelength of the incident beams.

Another important definition is the complex optical activity:

$$[o] = \sum_{i,j} C_{ij} A_{ij} \tag{2.8}$$

from which molar ellipticity and molar rotation are derived as the imaginary and negative real parts, respectively: $[\theta] = Im([o])$ and $[\phi] = -Re([o])$.

If we expand the matrix $\hat{\mathbf{A}}$, we obtain the series:

$$\hat{\mathbf{A}} = \hat{\mathbf{\Lambda}} - \hat{\mathbf{\Lambda}}\hat{\mathbf{G}}\hat{\mathbf{\Lambda}} + O[\hat{\mathbf{\Lambda}}^3], \tag{2.9}$$

Since the coefficients C_{ij} are zero when $i = j$, the first non-trivial contribution to $[o]$ is given by the second-order term of this expansion. Hence,

$$[o] \approx -\sum_{i,j} \alpha_i \alpha_j C_{ij} G_{ij}. \tag{2.10}$$

This level of approximation is generally sufficient to accurately describe the optical properties under consideration, as the subsequent terms in the expansion are quite small [37]. To obtain realistic spectra, the preceding equation must be combined with a rotational average.

Finally, the molar extinction coefficient, ε, which measures how strongly a chromophore absorbs, is given by:

$$\varepsilon = -\frac{8\pi^2 N_A}{3 \times 10^3 ln(10)\lambda} \sum_{i,j} \mathbf{e}_i \cdot \mathbf{e}_j Im(A_{ij}) \tag{2.11}$$

The primary contribution to this sum comes from the first-order terms, leading to the approximation $\varepsilon \sim \sum_i Im(\alpha_i)$. Since ε is a positive definite quantity, the imaginary part of the polarizability, $Im(\alpha_i)$, must be negative.

2.2 The Knowledge-Based Model of Circular Dichroism (KCD)

The primary goal of the KCD model is to estimate a set of effective atomic polarizabilities that enable the computation of molar ellipticity, $[\theta]$, from a given protein's three-dimensional structure. Therefore, the connection between the variables of interest and the experimental spectroscopic information, as described by equation 2.10, serves as the theoretical reference for constructing the model. To estimate the set of atomic polarizabilities, several necessary steps must be taken, as explained in the following paragraphs.

To establish a set of N atomic polarizabilities, a comparable number of functions for optical activity, $[o]$, of experimental origin are necessary. For a given protein p, its corresponding optical activity, $[o_p]$, can be obtained by first using high-quality CD spectroscopic data from the Protein Circular Dichroism Data Bank (PCDDB) to determine its experimental molar ellipticity, $[\theta_p]$. This data is typically provided in units of molar circular dichroism, $\Delta\varepsilon$, which can be converted into molar ellipticity using the following expression [20]:

$$[\theta] = \frac{4.5 \times 10^3 ln(10)}{\pi} \Delta\varepsilon \approx 3298.2 \Delta\varepsilon. \tag{2.12}$$

Since there is generally not enough experimental data for molar rotation (or optical rotatory dispersion), a Kramers-Kronig (K-K) transformation is used to relate molar rotation with molar ellipticity [36]:

$$[\phi(\lambda)] = \frac{2}{\pi} P \int_0^\infty \frac{\lambda'[\theta(\lambda')]}{\lambda^2 - \lambda'^2} d\lambda', \tag{2.13}$$

where P indicates the Cauchy principal value and numerical calculations are performed

using an appropriate algorithm as found in the referenced literature [38].

Conformational information is also required for this purpose. Three-dimensional structure data from the Protein Data Bank (PDB) is used to determine the coefficients C_{ij}^{p} and G_{ij}^{p}, where the superscript p denotes the given protein.

Since the proteins in the PCDDB are not strictly identical to their corresponding PDB structures (due to variations in pH and temperature, incomplete crystallographic data, etc.), a direct solution of equation 2.10 is not practical. Instead, for a given protein p, a mean atomic polarizability $\langle\alpha_p\rangle$ is calculated under the assumption that the induced dipole moments depend only on the applied electric field, as described in equation 2.1. Therefore, the equation for the mean polarizability squared becomes:

$$\langle\alpha_p\rangle^2 = -\frac{[o_p]}{\langle\sum_{i,j} C_{ij}^{p} G_{ij}^{p}\rangle_{\Omega}}, \tag{2.14}$$

where $\langle\rangle_{\Omega}$ denotes a rotational average. This average is performed to account for the possible orientations of the solvated molecule relative to the applied electric field.

The rotational average is efficiently calculated using the SPIRAL method, as described in the referenced literature [39]. This method is noted for its efficiency in exploring angular configurations. In this approach, the rotation matrix is defined by the angles Θ and Φ, corresponding to L equally spaced points of a spherical spiral:

$$\begin{aligned} \Theta_l &= arccos\left(\frac{2l-1-L}{L}\right) \\ \Phi_l &= \sqrt{\pi L} \times arcsin\left(\frac{2l-1-L}{L}\right) \\ l &= 1, ..., L. \end{aligned} \tag{2.15}$$

For standard CD spectra calculations using this method, $L = 400$ is sufficient, and this value can be decreased without significant loss of accuracy.

There are two possible solutions to equation 2.14, but only the physical solution is chosen. This solution is the only continuous one that gives a negative imaginary part, $Im(\langle\alpha_p\rangle) < 0$, which ensures a positive molar extinction coefficient, as discussed in the previous section. The real part of $\langle\alpha_p\rangle$ can be obtained by a Kramers-Kronig transformation:

$$Re(\langle\alpha_p(\lambda)\rangle) = \frac{2\lambda^2}{\pi} P \int_0^\infty \frac{Im(\langle\alpha_p(\lambda')\rangle)}{\lambda'[\lambda'^2 - \lambda^2]} d\lambda'. \tag{2.16}$$

This allows the atomic complex polarizabilities to be expressed as:

$$\alpha_a = \sum_p k_{ap} Re(\langle\alpha_p\rangle) + i \sum_p k'_{ap} Im(\langle\alpha_p\rangle), \tag{2.17}$$

where the subscript a denotes a specific atom, and k_{ap} and k'_{ap} are real coefficients to be determined. The condition $k'_{ap} > 0$ is imposed to ensure the molar extinction coefficient, ε, remains positive. With these polarizabilities and the working hypotheses, the optical activity $[o_p]$ can be rewritten as:

$$[o_p] = -\sum_{a,a'} \alpha_a \alpha_{a'} \langle C^p_{aa'} G^p_{aa'} \rangle_\Omega, \tag{2.18}$$

which contains the same information as equation 2.10 (aside from the angular average). The sets of coefficients $\{k_{ap}\}$ and $\{k'_{ap}\}$ are determined using a Monte Carlo (MC) algorithm. This iterative approach is guided by several key considerations.

Initially, the KCD method was developed using a reference set of 37 non-homologous protein structures from the Protein Data Bank (PDB) and their corresponding CD spectra from the Protein Circular Dichroism Data Bank (PCDDB) [37]. This set was later expanded to 120 proteins [33], which significantly enhanced the model's accuracy. For this work, the KCD method used a set of 125 non-homologous proteins. For each protein, the addition of missing hydrogen atoms is made using the software Discovery Studio Visualizer [40].

These proteins are then classified using the STRIDE algorithm [41], which is known to be more accurate in assigning secondary structures than the traditional Define Secondary Structure of Proteins (DSSP) method [42]. This basis is divided into three groups based on their predominant secondary structure content:

- α-helical: a minimum content of 27% α-helices.
- β-structures: at least 30% β-strands.
- Coils: at least 51% of randomly oriented residues.

This division is designed to capture the unique effects of electronic transitions that are intrinsic to each secondary structure type.

A proposed set of 21 isotropic polarizabilities, α_a, originated from different conformations of the atoms H, C, N, O, and S is then estimated by performing different MC simulations for each basis group. The simulation scheme is as follows:

1. **Initialization:** Begin by assigning arbitrary constants to the initial sets of coefficients $\{k^0_{ap}\}$ and $\{k'^0_{ap}\}$, which define the initial set of polarizabilities $\{\alpha^0_a\}$.

2. **Simulation:** From the imaginary part of equation 2.18, you obtain the initial set of simulated molar ellipticities, $\{[\theta^{sim}_p]\}$.

3. **Comparison:** The simulated spectra are then compared to the corresponding experimental spectra, $\{[\theta^{exp}_p]\}$, from the PCDDB using the Normalized Absolute Deviation (NAD) metric, $\hat{\Delta}[\theta]$:

$$\hat{\Delta}[\theta] = \frac{\sum_i^M |[\theta^{exp}_i] - [\theta^{sim}_i]|}{\sum_i^M |[\theta^{exp}_i]|} \tag{2.19}$$

   where M is the total number of data points, corresponging to discrete ellipticity data from $\lambda_{min} = 188$ nm to $\lambda_{max} = 244$ nm. NAD is highly interpretable, as a value of 0.1 represents a 10% deviation from the experimental curve, and a value of 0 would indicate a perfect match.

4. **Iteration:** In the next stage of the simulation, either k_{ap} or k'_{ap} is randomly selected and modified. The related polarizability α_a is then updated, and the new set $\{[\theta^{sim}_p]\}$ is calculated, along with the updated NAD. If the convergence parameter, $\hat{\Delta}[\theta]$, decreases, the MC step is accepted; otherwise, it is rejected.

5. **Termination:** This process is repeated until $\hat{\Delta}[\theta]$ remains essentially unchanged.

The entire process is initiated multiple times with different random seeds. Each run consists of approximately 10^6 steps. The systems with the lowest $\hat{\Delta}[\theta]$ then undergo additional $\sim 10^8$ steps to select the final set of polarizabilities.

Once the Monte Carlo simulations are complete, the final atomic polarizability for a given protein, α_{ap}, is determined by a linear combination of the average polarizabilities per group and individual protein polarizabilities:

$$\alpha_{ap} = c_{gp}\alpha_{ag} + \sum_{b} c_{bp}\alpha_{ab} \tag{2.20}$$

Here, α_{ag} represents the group average polarizabilities, while α_{ab} corresponds to the atomic polarizabilities of individual proteins from the basis set. The term c_{gp} is a constant of order 1. However, for the present work, this term has been excluded as computational analysis showed it did not significantly contribute to the accuracy of the computed CD spectra. The weight constants, c_{bp}, therefore play a more relevant and significant role. These weights depend on the proximity of the target protein's secondary structure content to each protein in the basis set and are evaluated through the expression:

$$c_{bp} = 100 \exp\left(\frac{-50[|s_{\alpha b} - s_{\alpha p}| + |s_{\beta b} - s_{\beta p}| + |s_{cb} - s_{cp}| + |s_{ob} - s_{op}|]}{\tau}\right). \tag{2.21}$$

Here, s_α, s_β, s_c, and s_o represent the secondary structural content of α-helices, β-strands, coils, and other structures, respectively. The subscripts b and p denote the protein from the basis and the target protein, respectively, and τ is a normalization constant.

Finally, for atoms belonging to D-amino acid residues, the corresponding polarizabilities, $\{\alpha_{a*}\}$, are modeled by taking the opposite sign of the real part of the polarizabilities, $\{\alpha_a\}$.

2.3 Introduction to Neural Networks

In the preceding section, we explored how the KCD model predicts circular dichroism (CD) spectra by calculating suitable atomic polarizabilities using equation 2.20. The weight constants in this equation, which are determined by the rule of proximity (equation 2.21), have led to the KCD model's notable accuracy. However, this accuracy can be further enhanced by identifying a more effective method for determining these weight

constants, thereby yielding more precise atomic polarizabilities. Deep neural networks are a powerful machine learning approach that excel at discovering intricate relationships within data. By mimicking the learning processes found in biological organisms, they are ideally suited to find a new rule that could provide better weight constants for the KCD model. This section will introduce the fundamental concepts of neural networks to lay the groundwork for their application in our study.

2.3.1 Biological and Artificial Neural Networks

Learning in biological organisms is mediated by neurons, specialized cells that communicate with one another via axons and dendrites. The crucial connection points between these are called synapses, and learning occurs through changes in their connection strengths in response to external stimuli [43].

Analogously, artificial neural networks (ANNs) simulate this process using computational units, also called neurons, that are interconnected by numerical weights (see Figure 2.1). These weights serve a function similar to that of biological synapses, determining the strength and influence of the connections. The network "computes" a function by propagating input values from the input neurons through the hidden layers to the output neuron(s). The learning process involves iteratively adjusting these weights based on a set of training data pairs of known inputs and their corresponding outputs. This iterative refinement allows the network to gradually learn the underlying function. Over time, the network's ability to accurately compute this function for previously unseen inputs improves, a key goal known as model generalization [44].

The true strength of ANNs, especially deep networks, lies in their ability to combine multiple layers of these interconnected units. This hierarchical structure allows them to learn highly complex, non-linear functions that are often beyond the scope of traditional machine learning methods. The specific arrangement and combination of these units, known as the network's architecture, is critical to its performance and requires careful design and analysis [45]. From this point forward, we will use the term "neural networks" to refer specifically to artificial neural networks.

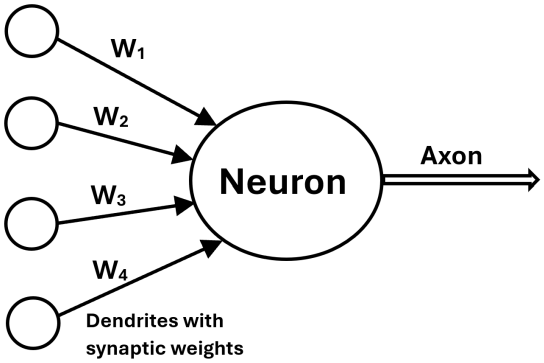


Figure 2.1: Schematic representation of an artificial neural network. Analogous to biological neural networks, the model consists of interconnected nodes (neurons) with inputs (dendrite-like), outputs (axon-like), and associated synaptic weights.

2.3.2 The Basics of Neural Networks

At its core, a neural network is a computational directed graph that maps a set of input nodes to a set of output nodes. Each node, or neuron, holds a variable, which is either an externally fixed input value or the result of a computation from preceding nodes. The connections between these nodes are represented by edges, each of which is assigned a numerical weight.

The network's architecture is organized into layers. In a single-layer network, inputs are directly connected to the output nodes. More complex multi-layer networks feature one or more hidden layers of nodes between the input and output layers. This layered arrangement allows the network to learn progressively more abstract representations of the input data. The final function computed by the network is a cascade of computations performed at each individual node. This architecture allows a neural network, with the appropriate set of weights, to approximate virtually any function that maps inputs to outputs.

The process of determining these weights in a data-driven manner is known as training. This process is framed as an optimization problem where the goal is to minimize the mismatch between the network's predicted output and the true target values from the training data. This mismatch is quantified by a loss function, which penalizes the

network for incorrect predictions.

For example, in a classification task with d inputs and a single binary output, each training instance is a pair $\left(\overline{X}, y\right)$, where $\overline{X} = [x_1, x_2, \ldots, x_d]$ is the input vector of d features and $y \in \{-1, +1\}$ is the corresponding target variable. The objective is to find the optimal weight vector $\overline{W}$ for a function $f_{\overline{W}}(\overline{X})$ that accurately predicts y, as shown by the relationship [44]:

$$y = f_{\overline{W}}(\overline{X}). \tag{2.22}$$

The subscript $\overline{W}$ denotes the explicit dependence of the function on the network's weights. The training process then involves an iterative adjustment of these weights to minimize the loss, ultimately enabling the network to generalize its predictions to new, unseen data.

2.3.3 The Perceptron

The simplest form of a neural network is the perceptron [46], which consists of a single input layer and a single output node (see Figure 2.2). Let's assume the input layer has d nodes, which transmit the features contained in the input vector $\overline{X} = [x_1, x_2, \ldots, x_d]$. The connections from these input nodes to the output node are weighted by a column vector $\overline{W} = [w_1, w_2, \ldots, w_d]^T$.

The input nodes themselves perform no computation; their role is solely to transmit the input values. The core computation occurs at the output node, which calculates the weighted sum of the inputs: $\overline{W} \cdot \overline{X}^T = \sum_{i=1}^{d} w_i x_i$. This value is then passed through a *sign* function, which acts as an activation function, converting the continuous value into a binary class label. This is particularly suitable for binary classification tasks. The perceptron's prediction, $\hat{y}$, is given by:

$$\hat{y} = f(\overline{X}) = sign(\overline{W} \cdot \overline{X}^T) = sign\left(\sum_{j=1}^{d} w_j x_j\right). \tag{2.23}$$

Here, the circumflex on $\hat{y}$ distinguishes it from the observed target value, y.

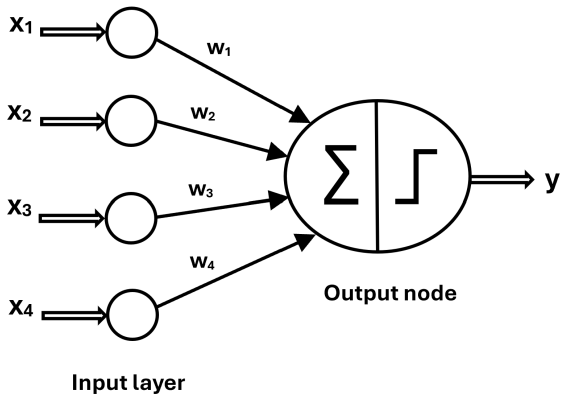


Figure 2.2: Schematic representation of a perceptron. The diagram illustrates the input layer with feature nodes x_i, each associated with a weight w_i, connected to the output node. The output node performs a weighted summation of the inputs, applies the *sign* activation function, and produces the binary output y.

Initially, the weights in $\overline{W}$ are often set randomly. The learning process then involves using the discrepancy between the predicted value $\hat{y}$ and the true value y to iteratively adjust these weights, improving the model's accuracy with each step. While the original perceptron algorithm used a heuristic update rule, it effectively implemented a form of gradient descent, a method commonly used today [46, 45].

The learning is formalized by the perceptron criterion, a loss function that penalizes misclassified instances. For a single training example $\left(\overline{X}_i, y_i\right)$, the loss L_i is defined as:

$$L_i = max\{-y_i(\overline{W} \cdot \overline{X}_i^T), 0\}. \tag{2.24}$$

This loss is non-zero only when the model's prediction, $sign\left(\overline{X}_i, y_i\right)$, differs from the true label y_i. To minimize this loss, we update the weights in the negative direction of the gradient, which indicates the direction of the steepest decrease in the loss. The gradient with respect to the weight vector $\overline{W}$ is given by:

$$\frac{\partial L_i}{\partial \overline{W}} = \left[\frac{\partial L_i}{\partial w_1}, ..., \frac{\partial L_i}{\partial w_d}\right]^T = \begin{cases} -y_i \overline{X}_i^T, & \text{if } sign(\overline{W} \cdot \overline{X}_i^T) \neq y_i \\ 0, & \text{otherwise.} \end{cases} \tag{2.25}$$

Consequently, the weight update rule for a misclassified instance is:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} = \overline{W} + \alpha y_i \overline{X}_i^T \tag{2.26}$$

where α is the learning rate, a hyperparameter that controls the size of the weight updates. This process is repeated for each training instance until the model converges or reaches a desired level of accuracy.

2.3.4 Basic Components of Neural Architectures

As we saw with the perceptron, the design of a neural network architecture is defined by more than just its layers and connections. The choice of activation function within a node and the loss function used during training are critical design components that significantly influence a model's performance. These choices, along with other configurable settings known as hyperparameters, are fundamental to building a successful network. This subsection will introduce some of the most common activation functions, loss functions, and key hyperparameters used in modern neural network design.

Activation Functions

An activation function, denoted by Φ, introduces non-linearity into a neural network. For a single-layer network with a weight vector $\overline{W}$ and an input vector $\overline{X}$, the prediction $\hat{y}$ is given by:

$$\hat{y} = \Phi(\overline{W} \cdot \overline{X}^T). \tag{2.27}$$

The most straightforward activation function is the identity function, or linear activation, $\Phi(u) = u$. This function is typically used in the output layer of a network designed for regression tasks where the target variable is a continuous real number.

The real power of multi-layer neural networks, however, comes from using non-linear activation functions. A network with only linear activation functions, regardless of the number of layers, can always be simplified to a single-layer network, limiting its ability to learn complex relationships [44]. Non-linearities are thus essential for a network to approximate any arbitrary function.

Historically, the most common non-linear activation functions were the sign, sigmoid, and hyperbolic tangent (tanh) functions:

$$\begin{aligned} \Phi(u) &= \text{sign}(u), \text{ (sign)} \\ \Phi(u) &= \frac{1}{1+e^{-u}}, \text{ (sigmoid)} \\ \Phi(u) &= \frac{e^{2u}-1}{e^{2u}+1}, \text{ (tanh).} \end{aligned} \tag{2.28}$$

While the sign function is useful for binary classification, its non-differentiable nature makes it unsuitable for gradient-based training methods. In contrast, the sigmoid and tanh functions are continuously differentiable, making them ideal for algorithms like gradient descent. These functions are often referred to as squashing functions because they map an input from an arbitrary range to a bounded one.

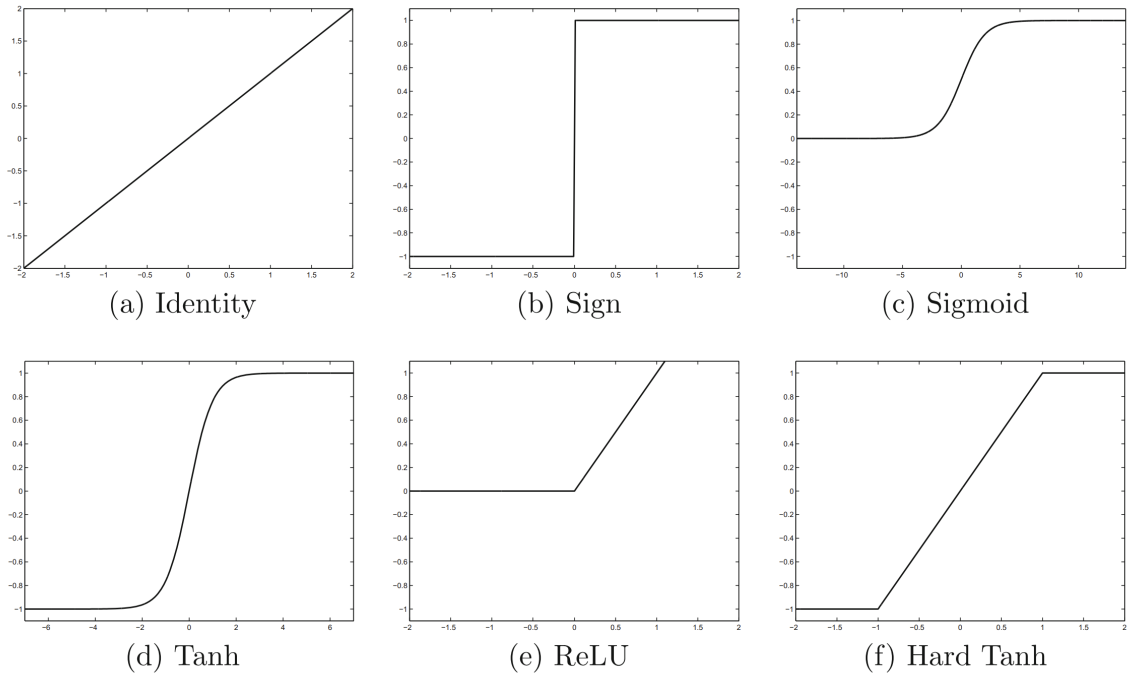
In recent years, the Rectified Linear Unit (ReLU) and its variants have largely replaced sigmoid and tanh due to their computational efficiency and ability to mitigate the vanishing gradient problem [47, 48]. The ReLU and hard tanh functions are defined as:

$$\begin{aligned} \Phi(u) &= \max\{u, 0\}, \text{ (ReLU)} \\ \Phi(u) &= \max\{\min[u, 1], -1\}, \text{ (hard tanh).} \end{aligned} \tag{2.29}$$

The graphs of these activation functions are illustrated in Figure 2.3. The simplicity and effectiveness of ReLU, in particular, have made it the default choice for hidden layers in many modern deep learning architectures. Accordingly, it is the activation function employed in this work.

The softplus activation function

While the Rectified Linear Unit (ReLU) has been one of the most successful activation functions in deep learning, as discussed previously, it is susceptible to the dying ReLU problem. This issue, a form of vanishing gradient, occurs when neurons become inactive, outputting zero for all subsequent inputs, which hinders the training of deep feed-forward



(a) Identity (b) Sign (c) Sigmoid

(d) Tanh (e) ReLU (f) Hard Tanh

Figure 2.3: Plots of commonly used activation functions in neural networks. (a) Identity function, producing a linear output equal to the input; (b) Sign function, yielding discrete outputs based on the sign of the input; (c) Sigmoid function, mapping inputs to values in the interval [0,1]; (d) Hyperbolic tangent (Tanh) function, mapping inputs to the range [-1,1]; (e) Rectified Linear Unit (ReLU); and (f) Hard Tanh, a piecewise linear approximation of the Tanh function bounded between -1 and 1 [44].

networks [49]. To mitigate this obstacle, various alternative methods and activation functions have been proposed, one of which is the Softplus function.

The Softplus function serves as a smooth, continuously differentiable approximation to the ReLU function. It is mathematically defined as:

$$\Phi(u) = \ln(1 + e^u). \tag{2.30}$$

The key advantage of Softplus is its smooth derivative, which is mathematically equal to the sigmoid function $\left(\frac{d}{du}\ln(1+e^u) = \frac{e^u}{1+e^u} = \text{sigmoid}(u)\right)$. This continuous differentiability is better suited for the backpropagation algorithm, ensuring smoother gradient updates and potentially more stable training than ReLU. Although a network trained with ReLU can achieve local minima of comparable quality, the smooth nature of Softplus provides a valuable advantage in optimization [50]. For these reasons, Softplus is the output activation function selected for the deep neural networks implemented in this work.

Loss Functions

The choice of a loss function is crucial and depends heavily on the specific application of the neural network. For regression problems, where the objective is to predict continuous numerical values, the following are common choices [51]:

1. **Mean Absolute Error (MSE):** The MAE, also known as L1 loss, measures the average of the absolute differences between the predicted and target values. For N output predictions, it is defined as:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{2.31}$$

   While straightforward and intuitive, a key challenge with MAE is that its derivative is undefined at the point where the error is zero, which can complicate gradient-based optimization algorithms.

2. **Mean Squared Error (MSE):** The MSE, or L2 loss, calculates the average of the squared differences between the predicted and target values. It is defined as:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{2.32}$$

Because it squares the error, MSE penalizes large errors more heavily than small ones. This characteristic makes it highly sensitive to outliers. A major advantage of MSE is that it is a continuously differentiable function, which makes it well-suited for gradient-based optimization algorithms. Since the outputs of the neural network are the continuous, real-valued weight constants used for calculating atomic polarizabilities, MSE is the appropriate loss function implemented in this work to minimize the deviation of the predicted weights from their optimal values.

Hyperparameters

Hyperparameters are external settings that regulate the design and training of a neural network model. Unlike the weights, which are learned from the data, hyperparameters must be set by the user prior to training [45]. They can be architectural and training-related.

1. **Architectural Hyperparameters**

   - **Number of layers and neurons per layer:** These two parameters dictate the overall complexity and capacity of the network, with deeper and wider networks generally having the potential to learn more complex functions.
   - **Weight initialization:** The initial values assigned to the network's weights can significantly impact the training process, as poor initialization can lead to slow convergence or unstable gradients.
   - **Dropout:** This is a regularization technique used to prevent overfitting, a state where the model learns the training data too well, but performs poorly on new, unseen data. During training, dropout randomly deactivates a subset of neurons, forcing the network to learn more robust and independent features

and preventing it from becoming overly reliant on any single neuron or specific pathway.

2. **Training-related Hyperparameters**

   - **Learning rate:** This parameter determines the step size for weight updates during each iteration of the gradient descent algorithm. A learning rate that is too high can cause the model to overshoot the optimal solution, while one that is too low can lead to slow convergence.
   - **Number of epochs:** An epoch represents a single pass through the entire training dataset. The number of epochs determines how many times the network is exposed to the full set of training examples.
   - **Batch size:** This specifies the number of training samples processed in each iteration before the model's weights are updated. Using a larger batch size provides a more accurate estimate of the gradient but requires more memory, while a smaller batch size can introduce more noise into the updates, but may help the model escape local minima.

Hyperparameter Optimization

Neural networks are highly sensitive to their hyperparameter configuration, and finding the optimal set of values for a given problem, a process known as fine-tuning, can be a computationally intensive task. To address this, various hyperparameter optimization (HO) techniques have been developed to automate the search for the best model settings. Common methods include grid search, random search, and Bayesian optimization [52]:

1. **Grid Search:** In grid search, the user defines a discrete set of values for each hyperparameter. The algorithm then exhaustively evaluates the model's performance for every possible combination of these values. While simple and guaranteed to find the best combination within the defined grid, this method becomes computationally intractable as the number of hyperparameters or the size of their value ranges increases. A common strategy to mitigate this is to perform a coarse grid search

initially to identify a promising region, followed by a finer-grained search within that region.

2. **Random Search:** Random search addresses the inefficiency of grid search by sampling hyperparameter combinations from specified distributions instead of a predefined grid. This approach is often more effective, as it allows for a broader exploration of the search space with the same number of evaluations, making it more likely to find a better set of hyperparameters, particularly when only a few of them are critical to performance.

3. **Bayesian Optimization:** Bayesian optimization is a more sophisticated and efficient method that leverages Bayes' theorem to guide the search process. It models the performance of the model as a function of the hyperparameters and uses this model to intelligently choose the next set of hyperparameters to evaluate. The process involves two key components: a surrogate model (often a Gaussian Process) to predict model performance and an acquisition function to determine the next most promising set of hyperparameters to test. By balancing exploration (testing in uncertain regions) and exploitation (testing in regions known to perform well), Bayesian optimization can converge to an optimal set of hyperparameters much more quickly than grid or random search, especially in high-dimensional and complex search spaces. Therefore, this is the method used in this work.

2.3.5 Multilayer Neural Networks

As previously noted, increasing the number of layers in a neural network enhances its capacity to learn more complex relationships within the data. This subsection delves into the fundamental concepts of these more sophisticated architectures.

Multilayer neural networks, often referred to as feed-forward networks, contain multiple computational layers. The layers positioned between the input and output layers are called hidden layers because their computations are not directly exposed to the user. In the standard feed-forward architecture, every node in one layer is connected to all nodes in the subsequent layer. An illustrative example of a multilayer network is

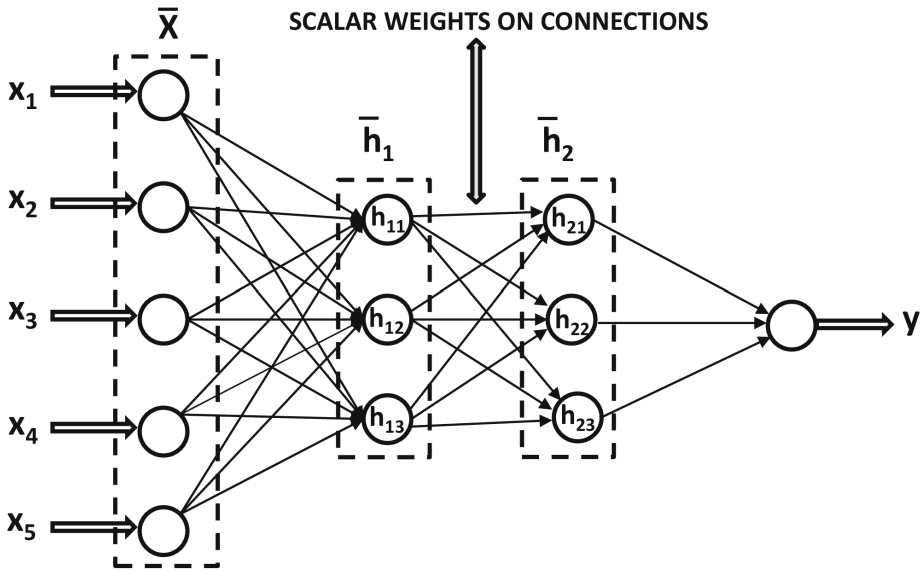


Figure 2.4: Schematic representation of a multilayer perceptron (MLP) with two hidden layers. The input layer, enclosed in a dotted rectangle, consists of the feature vector $\overline{x} = [x_1, ..., x_5]$. Hidden layer 1, also represented within a dotted rectangle, corresponds to the vector $\overline{h}_1 = [h_{11}, h_{12}, h_{13}]$, while hidden layer 2 is given by $\overline{h}_2 = [h_{21}, h_{22}, h_{23}]$. The network output is a single neuron producing the scalar value y [44].

shown in Figure 2.4.

For a network with d input units, k hidden layers, and p_r units in the rth hidden layer, the outputs of these hidden layers are represented by column vectors $\overline{h}_1, ..., \overline{h}_k$ with dimensionalities $p_1, ..., p_k$, respectively.

The connections between layers are governed by weight matrices:

- The matrix W_1 of size $p_1 \times d$ contains the weights connecting the input layer to the first hidden layer.
- For subsequent hidden layers, the matrix W_{r+1} of size $p_{r+1} \times p_r$ contains the weights between the rth and the $(r + 1)$th hidden layers.
- Finally, the matrix W_{k+1} of size $o \times p_k$ contains the weights from the last hidden layer to the output layer, which has o nodes.

The network transforms the d-dimensional input vector $\overline{x}$ into the final output vector $\overline{o}$ through a series of recursive matrix-vector multiplications and element-wise activation functions. This process is defined as follows:

$$\overline{h}_1 = \Phi(W_1\overline{x}) \qquad \text{Input to Hidden Layer} \qquad (2.33)$$

$$\overline{h}_{p+1} = \Phi(W_{p+1}\overline{h}_p) \quad \forall p \in \{1, ..., k-1\} \qquad \text{Hidden to Hidden Layer} \qquad (2.34)$$

$$\overline{o} = \Phi(W_{k+1}\overline{h}_k) \qquad \text{Hidden to Output Layer} \qquad (2.35)$$

Note that the activation function Φ is applied element-wise to the vector result of the matrix multiplication, creating an output vector of the same dimension where each element has been transformed by the function.

2.3.6 Structured Data and Advanced Architectures

The effectiveness of a neural network heavily depends on matching its architecture to the structure and characteristics of the input data. A domain-specific understanding of the data is therefore crucial for designing or selecting an appropriate specialized neural architecture [53]. Several well-established architectures are commonly employed to address different data types, and are summarized in Table 2.1.

In this work, the input features that determine the weight constants for atomic polarizabilities are represented as feature vectors lacking any inherent spatial or temporal dependencies. Therefore, our data is fundamentally tabular, a structure that can be effectively addressed by Bayesian Neural Networks (BNN), which will form the basis of the networks implemented in this study.

2.4 Bayesian Neural Networks

The remarkable success of Deep Learning in extracting complex features and patterns has transformed many traditionally difficult fields. However, these models face challenges, notably their tendency toward overfitting and their overconfidence when providing predictions. In high-stakes applications like autonomous driving, medical diagnosis, or finance, where silent failures can lead to dramatic outcomes, accurately quantifying the uncertainty of a prediction is paramount [54]. The Bayesian paradigm offers a robust

Table 2.1: Neural Network Architectures Tailored to Data Structure [44, 53].

Data Type	**Characteristics**	**Standard Architecture**
Sequential Data	Order and dependencies matter; variable length (e.g., text, time series, DNA sequences).	Recurrent Neural Networks (RNNs), LSTMs, GRUs.
Image/Vision Data	High dimensionality; strong spatial structure; translation and rotation invariant (e.g., photographs, medical scans).	Convolutional Neural Networks (CNNs), utilizing the convolution operation.
Graph Data	Networked entities represented as nodes and edges (e.g., social networks, molecular structures).	Graph Neural Networks (GNNs).
Tabular Data	Unstructured collections of feature vectors without inherent spatial or temporal relationships (e.g., spreadsheets, databases).	Multi-layer Perceptrons (MLPs) and Bayesian Neural Networks.

mathematical framework to address this challenge by quantifying uncertainty in deep learning models.

2.4.1 The Bayesian Framework

At the core of the Bayesian paradigm is Bayes' Theorem, which provides a mechanism to update the belief in a hypothesis (H) given new data (D) [54, 55]:

$$P(H|D) = \frac{P(D|H)P(H)}{P(D)} = \frac{P(D,H)}{\int_H P(D,H')dH'}, \tag{2.36}$$

here:

- $P(H)$ is the Prior probability, representing the initial belief about the hypothesis.
- $P(D|H)$ is the Likelihood, which encodes the model's aleatoric uncertainty (the uncertainty inherent in the data itself, such as noise).
- $P(D)$ is the Evidence (a normalization constant).
- $P(H|D)$ is the Posterior probability, representing the updated belief after seeing the data, which encapsulates the epistemic uncertainty (the uncertainty due to lack of data or knowledge of the model parameters).

2.4.2 Aplication to Neural Networks

Bayesian Neural Networks (BNNs) are stochastic neural networks that leverage Bayesian inference [54]. Instead of learning a single set of optimal weights (a "point estimate"), BNNs treat the weights, $\boldsymbol{\theta}$, as random variables with associated probability distributions, $p(\boldsymbol{\theta})$ (see Figure 2.5). This process simulates an ensemble of multiple possible models. Just as an ensemble of independent, average-performing predictors can outperform a single well-performing model, BNNs provide robust predictions and, crucially, quantify the model's inherent uncertainty. If the ensemble of models agrees on a prediction, the uncertainty is low; if they disagree, the uncertainty is high.

For a BNN, the model parameters ($\boldsymbol{\theta}$, i.e., the network's weights) are the hypotheses we seek to update. The training inputs D_x and labels D_y form the data D. Applying Bayes' theorem yields the Bayesian posterior distribution over the weights [54]:

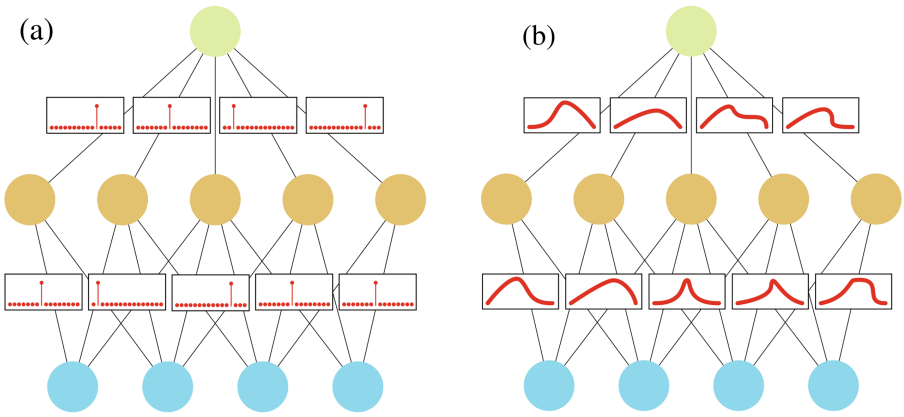


Figure 2.5: (a) Point estimate neural network, (b) Bayesian neural network with a probability distribution over the weights [54].

$$p(\boldsymbol{\theta}|D) = \frac{p(D_y|D_x, \boldsymbol{\theta})p(\boldsymbol{\theta})}{\int_{\boldsymbol{\theta}} p(D_y|D_x, \boldsymbol{\theta}')p(\boldsymbol{\theta}')d\boldsymbol{\theta}'} \propto p(D_y|D_x, \boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{2.37}$$

This posterior distribution, $p(\boldsymbol{\theta}|D)$, is the ultimate goal of the training process, as it represents the possible distribution of the network's weights after observing the training data.

For prediction on a new input $\boldsymbol{x}$, the uncertainty-aware prediction is given by the posterior predictive distribution, $p(\boldsymbol{y}|\boldsymbol{x}, D)$ [54]:

$$p(\boldsymbol{y}|\boldsymbol{x}, D) = \int_{\boldsymbol{\theta}} p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}')p(\boldsymbol{\theta}'|D)d\boldsymbol{\theta}'. \tag{2.38}$$

This marginal distribution integrates over all possible weight configurations, quantifying the model's total predictive uncertainty. In practice, this integral is approximated by sampling predictions from the trained BNN using weights drawn from the posterior distribution and collecting statistics such as the mean (the prediction) and the standard deviation (the uncertainty). The BNN implemented in this work utilizes a multilayer perceptron architecture with stochastic weights to achieve this uncertainty quantification.

Chapter 3

Methodology

In this chapter, we explain the process of implementing deep Bayesian neural networks (BNNs) within the KCD framework. The methodology is presented in five stages: (1) data collection and preprocessing; (2) an initial weight optimization for each protein using a multilayer perceptron (MLP) to establish a refined baseline; (3) data augmentation to ensure robust model training; (4) the tuning and training of the BNN architectures; and (5) the integration with the KCD model.

3.1 Data Collection and Preprocessing

We used the learning set of KCD, consisting of 125 experimental spectra from the PCDDB that are identified with and index number (Index) and their related PDB code (PDB ID), (see Table 3.1). This set will be used to construct the training and validation sets. Additionally, we used a separate test set consisting of 50 proteins from the PCDDB, which can be viewed in Table 3.2.

It is important to note that, as stated in preceding chapters, although the PCDDB contains 741 deposited spectra, not all of them were suitable for this work. Some proteins in the PCDDB have repeated corresponding spectra, limiting the statistical analysis to a single useful spectrum. Examples of this include the sodium/potassium-transporting ATPase (280 entries), ion transport protein (55 entries), BH1501 Protein (49 entries), NavMs Full Length (18 entries), Translocate Actin Recruiting Phosphoprotein (17 entries), Lysozyme (17 entries), MEG-14 (10 entries), EGFP (5 entries), E5 (6 entries), Bovine Collagen Type II (5 entries), Formate Oxidase (5 entries), and Presenilin 2 (4 entries), among others. Other excluded spectra include those that were noisy or non-existent for wavelengths smaller than 195 nm, and cases in which structural information

Table 3.1: KCD learning set

Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID
1	1n5u	26	1trz	51	1xag	76	1sfl	101	5cha
2	1lin	27	1stp	52	1ado	77	1bn8	102	4kyp
3	2cts	28	4gcr	53	1hrc	78	2psg	103	1nqh
4	7atj	29	1ubi	54	2j0x	79	1uyn	104	1t5s
5	1une	30	3rn3	55	2dhq	80	1azu	105	3est
6	2y3z	31	1auj	56	1a49	81	1v9e	106	1a0s
7	1qfe	32	1ba7	57	2gel	82	2cga	107	1ku8
8	7tim	33	1jpc	58	3pmg	83	2pab	108	1blf
9	1bn6	34	1k9b	59	1hnn	84	1thw	109	6y84
10	5cpa	35	1r0i	60	1hml	85	9wga	110	2ccm
11	1rhs	36	2fdn	61	1fa2	86	1elp	111	1dot
12	3pgk	37	3gny	62	2j51	87	2rhe	112	1gpb
13	1cf3	38	1qhj	63	1xl4	88	1bd7	113	1fcp
14	1ppn	39	2oar	64	1dgf	89	1hk0	114	1ha7
15	1mol	40	2a65	65	1scd	90	5pti	115	1igt
16	1air	41	2nop	66	1a8e	91	2yxf	116	1hc9
17	1a7h	42	1nkz	67	1ova	92	1rav	117	1hcb
18	1ha4	43	1hda	68	1qlp	93	1kcw	118	1a6m
19	2bb2	44	2cfq	69	1ed9	94	1kit	119	1ies
20	1cbj	45	1j95	70	1vjs	95	3njw	120	3jqo
21	1m8u	46	1hzx	71	3dni	96	1les	121	3qtk
22	1jbc	47	2pjf	72	1wnh	97	1ofs	122	2gif
23	1ymb	48	2nr9	73	2sns	98	1fep	123	1k6j
24	194l	49	1gai	74	5dfr	99	2e8d	124	2wjn
25	1ecz	50	6dhm	75	1hk9	100	1t4b	125	1nek

Table 3.2: Test set

Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID
1	1qi7	11	1b8e	21	2ox0	31	2kpk	41	2wcb
2	1rh5	12	2j58	22	4v40	32	3qtk	42	2nwe
3	2vdf	13	1kpk	23	2zxe	33	4p9o	43	4f4l
4	3q9t	14	1q5u	24	1ax8	34	3kdp	44	q9avb0
5	5a37	15	5hvd	25	4p90	35	2hav	45	p49810
6	5hvx	16	3n23	26	5jhe	36	1h2s	46	p07148
7	5tav	17	1pcr	27	2iwv	37	1sr5	47	q9buz4
8	2m9g	18	3qlp	28	2dyr	38	2j41	48	q9m0n
9	6lhm	19	1l7v	29	1be3	39	1oki	49	p62568
10	2kit	20	1mdu	30	3go0	40	1ytq	50	p69905

was absent, such as for CGlbN, Immunoglobulin G1, and Amelogenins.

3.2 KCD Weight Optimization

For each protein from both the learning set and the test set, an optimization of the KCD constant weights was performed using a gradient descent-based approach. This per-protein optimization consists of the following steps:

1. Run KCD for the protein to obtain the initial weight set of constants. Polarization data is generated via a linear mixing of these weights, which allows for the calculation of the predicted spectrum and its associated Normalized Absolute Deviation (NAD).

2. A multilayer perceptron (MLP) is employed, using the KCD-computed NAD as its loss function. The original KCD weight constants file is fed as input to the network, and the network outputs a modified (optimized) set of weight constants. The MLP architecture consists of two hidden layers, each with 200 neurons, using the softplus activation function.

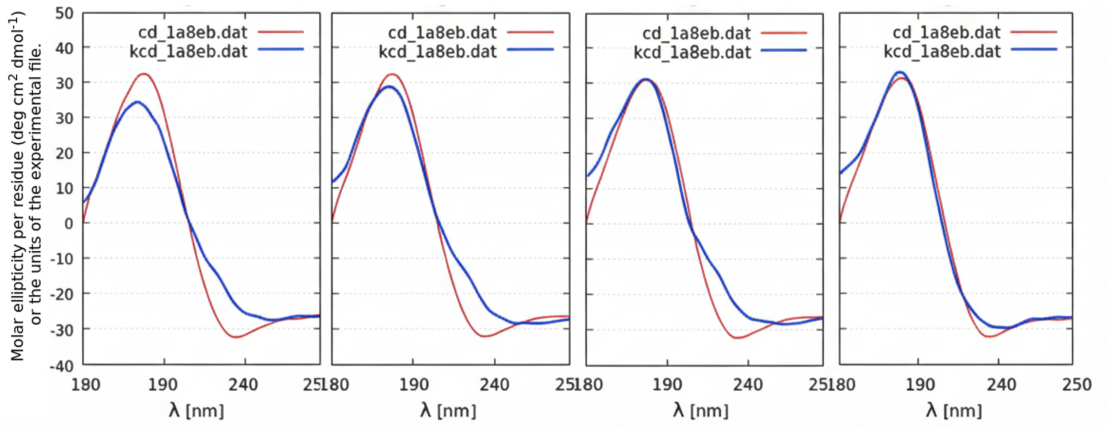


Figure 3.1: Four stages of the KCD weight optimization for protein 1a8e. The KCD calculated spectrum (blue line) is shown at different stages as it is iteratively fitted to the experimental spectrum (red line). The optimization process refines the weight constants to minimize the deviation between the predicted and experimental spectra.

3. A multilayer perceptron (MLP) is employed, using the KCD-computed NAD as its loss function. The original KCD weight constants file is fed as input to the network, and the network outputs a modified (optimized) set of weight constants. The MLP architecture consists of three hidden layers, each with 125 neurons.

4. The network is trained using gradient descent until the NAD-based loss function converges. Convergence is defined as the point where the change in the loss value between epochs is less than 0.001.

5. A new, optimized weight set of constants is obtained for this specific protein.

This optimization procedure was applied to all proteins in the learning and test sets, yielding an optimized KCD weight file for each entry. Some stages in the optimization process can be viewed in Figure 3.1.

3.3 Data Augmentation

It is worth noting that the power of deep learning models has been demonstrated in the big data era, where they have overperformed traditional machine learning algorithms given the availability of larger datasets (see Figure 3.2). Therefore, for a problem with

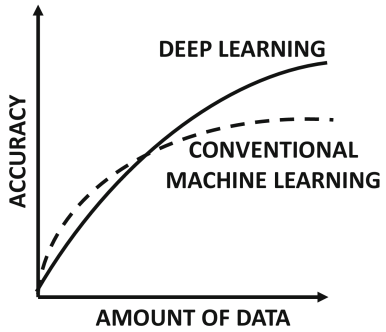


Figure 3.2: Relationship between data availability and model accuracy for traditional machine learning and deep learning approaches. The deep neural network shows superior performance as the amount of training data increases [44].

limited data like ours, data augmentation is essential to achieve good performance from our deep learning models.

Data augmentation is a technique where the training data is transformed into new synthetic data, allowing the network to be trained with a larger data set (training + augmented) and ensuring robust model training. Such a transformation is made in this work to fill gaps in the four-dimensional space of protein secondary structure (α-helix, β-sheet, coils, and others). For example, in a hypothetical 3D space (Figure 3.3) composed of only α-helix, β-sheet, and coil content, the proteins of the KCD learning set would correspond to points in that space. Augmented data would then correspond to new combinations of these three structure types, for which a new set of weight constants would be interpolated.

This process consists of two primary stages: (1) generating the synthetic 4D secondary structure inputs, and (2) generating the corresponding synthetic 125-dimensional weight constant outputs.

First, new synthetic inputs were created by generating combinations of the four secondary structure contents, such that the sum of the four components is one, and the new points are homogenously distributed throughout the feature space.

Second, to create the synthetic weights for these new inputs, we developed a generative model based on the optimized weights of the learning set. This was built by first analyzing the relationships within the original 125-protein set. We calculated all

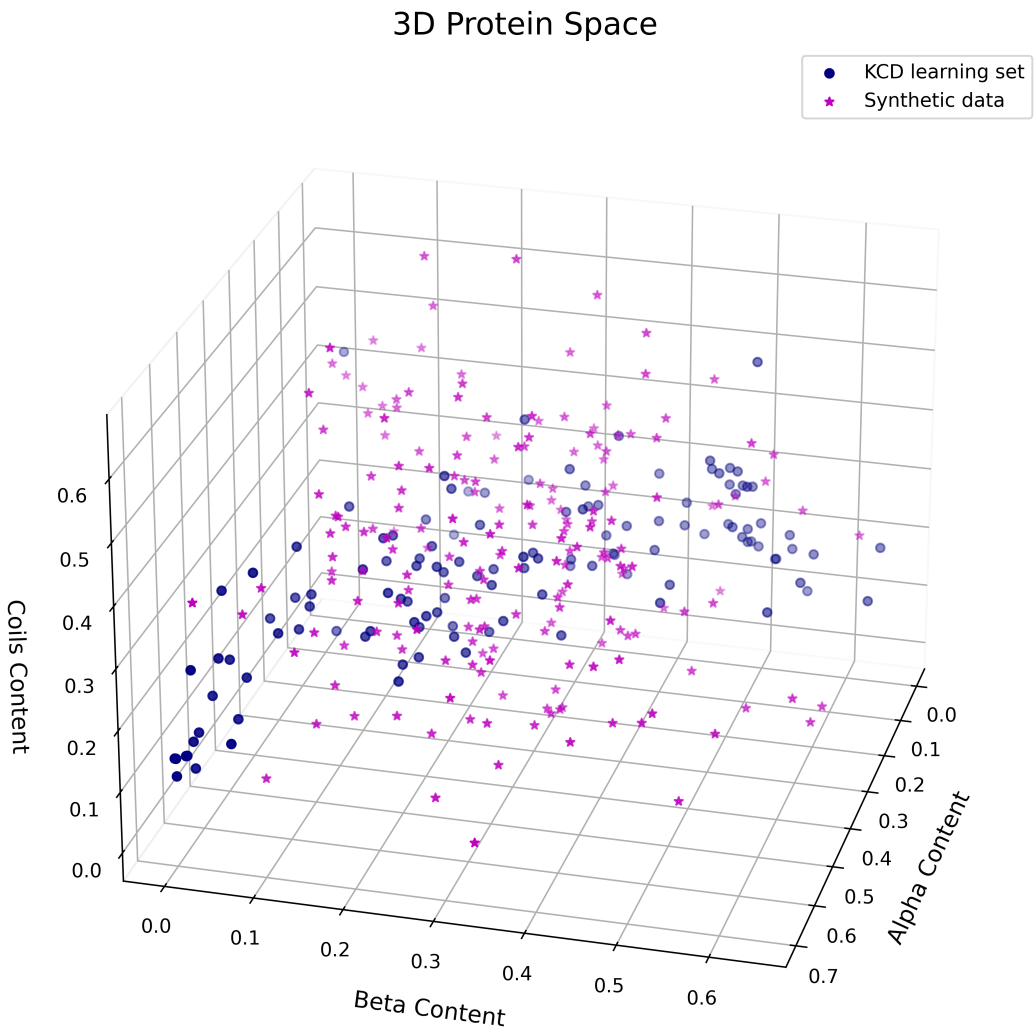


Figure 3.3: Visualization of hypothetical 3D protein structural space showing the distribution of α-helix, β-sheet, and coil content. The plot compares natural proteins from the KCD learning set (navy points) with generated synthetic data (magenta stars).

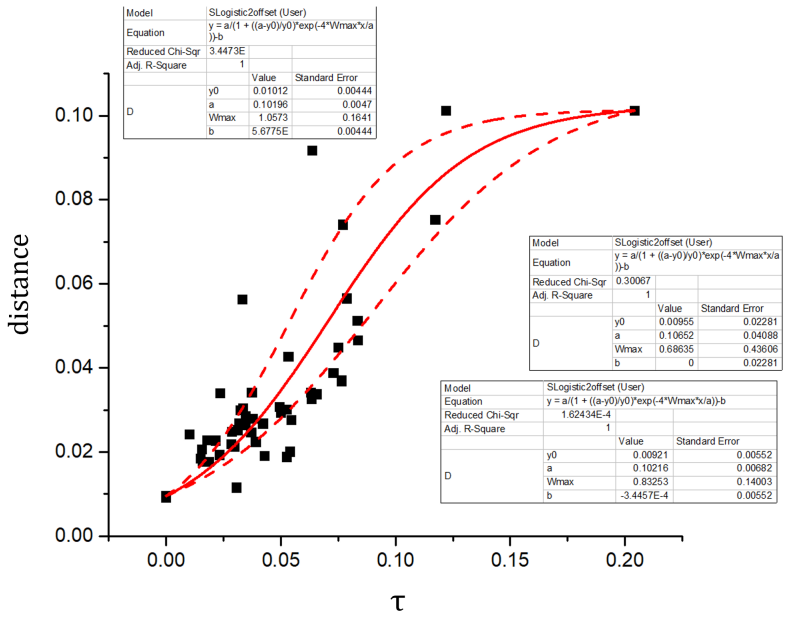


Figure 3.4: The parametric curve used to model the relationship between protein features and weights for data augmentation. The Y-axis plots the minimum Euclidean distance (d_{min}) to its nearest neighbor in the 4D secondary structure space, while the X-axis plots the corresponding τ parameter. The solid red line shows the 'SLogistic2offset' function fitted to this data. This fitted function serves as the generative model for creating synthetic weights. The dashed lines represent the confidence bands of the fit.

Euclidean distances d between proteins in the 4D feature space and, for each protein, identified the minimum distance to its nearest neighbor, d_{min}. This d_{min} value was then used to construct the parametric curve shown in Figure 3.4. This curve, which models the relationship between protein similarity (in 4D space) and their corresponding weights (in 125D space), was then fitted using the Origin software function 'SLogistic2offset', which provided the best R^2 coefficient.

With this fitted logistic function acting as our generative model, we could input any of the newly created synthetic 4D secondary structure proteins and generate a full set of 125 synthetic weight constants.

Using this method, we created packages of 10,000 synthetic data points. These packages were used for training, while the original 125-protein learning set was kept separate for validation.

3.4 Neural Network Architecture and Training

To handle the complexity of predicting 125 weight constants from only 4 feature inputs, our neural architecture design began with a K-means clustering step to divide the problem space. The 125-protein learning set was divided into four groups; this technique consists of grouping data without prior information on the group, which allows a specialized network to be trained for each resulting group.

For this task, we used the K-means machine learning algorithm implemented in scikit-learn [56], which consists of the following steps:

1. Randomly initialize K feature mean vectors, in this case K=4.
2. Calculate for each protein the Euclidean distance (taking as argument the features, i.e., the secondary structure content) between the protein and the mean vectors.
3. Assign each protein to the cluster with the minimum of such distances.
4. Calculate the mean vector for each of the four updated clusters and update it.
5. Repeat steps 2-4 until there is no change in the mean vectors.
6. Obtain the four clusters of proteins.

With this algorithm, we ensure we have four clusters of proteins sharing similar features, i.e., similar secondary structure contents, as seen graphically in Figure 3.5. A list of proteins belonging to each cluster can be found in Tables 3.3, 3.4, 3.5, and 3.6.

With these four clusters defined, we designed a "divide-and-conquer" architecture. Instead of one large BNN predicting all 125 weight constants, we trained four smaller, specialized BNNs.

The clustering step was not used to route input proteins; rather, it was used to structurally divide the 125-constant output vector. Each BNN was assigned a fixed, static subset of the 125 weight constants to predict.

This assignment was determined by the cluster membership of the original 125 proteins in the KCD learning set (b_j, where $j = 1...125$). For example, if the learning

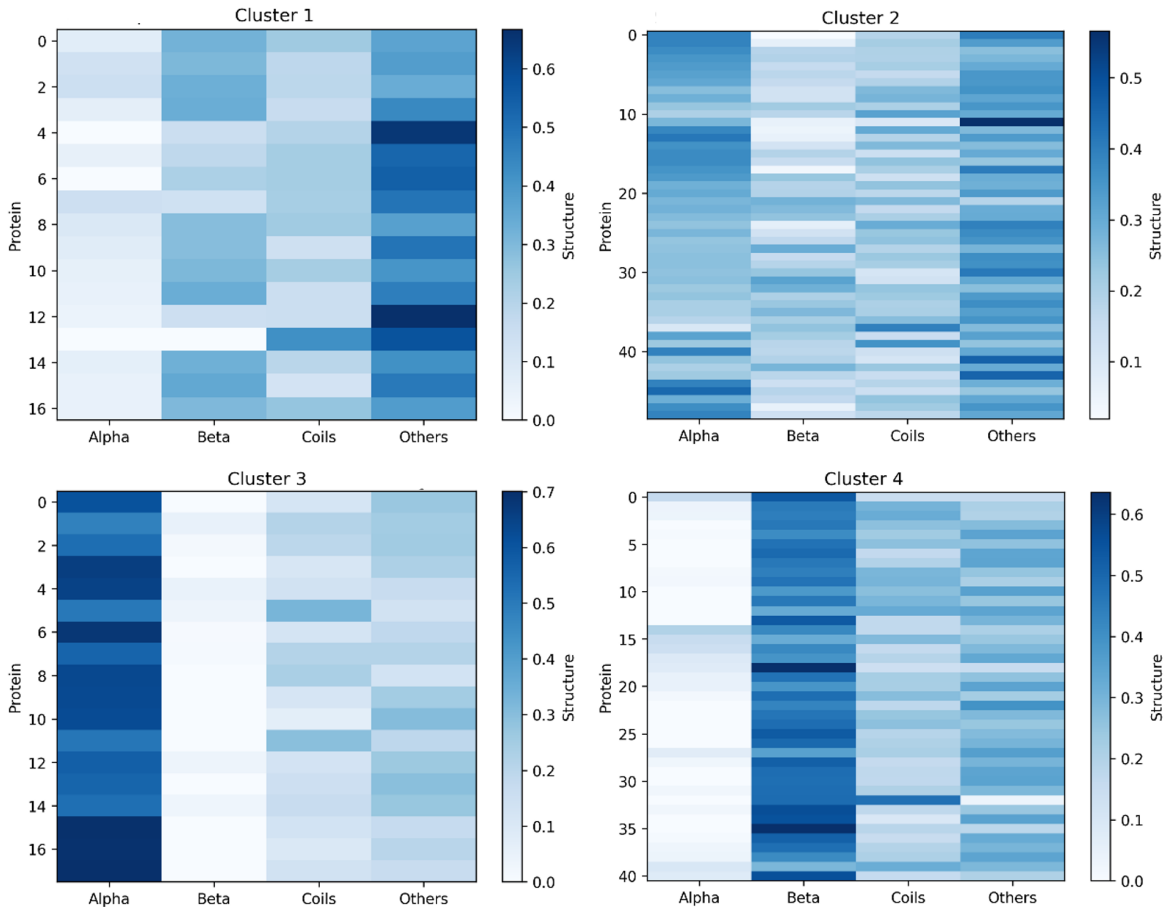


Figure 3.5: Secondary structure content across the four protein clusters. Each heatmap represents one cluster. The Y-axis lists the proteins belonging to that cluster, while the X-axis shows the four types of secondary structure: Alpha, Beta, Coils, and Others. The color scale (from light to dark blue) indicates the relative content of each secondary structure type, ranging from 0 to 1.

Table 3.3: Learning Set Proteins in Cluster 1

Index	PDB ID	Index	PDB ID	Index	PDB ID
16	1air	47	2pjf	85	9wga
29	1ubi	76	1sfl	95	3njw
30	3rn3	77	1bn8	101	5cha
31	1auj	80	1azu	105	3est
35	1r0i	81	1v9e	117	1hcb
36	2fdn	82	2cga		

Table 3.4: Learning Set Proteins in Cluster 2

Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID
4	7atj	14	1ppn	56	1a49	66	1a8e	104	1t5s
5	1une	24	194l	57	2gel	67	1ova	108	1blf
6	2y3z	26	1trz	58	3pmg	68	1qlp	109	6y84
7	1qfe	49	1gai	59	1hnn	69	1ed9	111	1dot
8	7tim	50	6dhm	60	1hml	70	1vjs	112	1gpb
9	1bn6	51	1xag	61	1fa2	71	3dni	122	2gif
10	5cpa	52	1ado	62	2j51	73	2sns	123	1k6j
11	1rhs	53	1hrc	63	1xl4	90	5pti	124	2wjn
12	3pgk	54	2j0x	64	1dgf	100	1t4b	125	1nek
13	1cf3	55	2dhq	65	1scd	102	4kyp		

Table 3.5: Learning Set Proteins in Cluster 3

Index	PDB ID	Index	PDB ID	Index	PDB ID
1	1n5u	40	2a65	46	1hzx
2	1lin	41	2nop	48	2nr9
3	2cts	42	1nkz	110	2ccm
23	1ymb	43	1hda	114	1ha7
38	1qhj	44	2cfq	118	1a6m
39	2oar	45	1j95	119	1ies

Table 3.6: Learning Set Proteins in Cluster 4

Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID	Index	PDB ID
15	1mol	28	4gcr	79	1uyn	93	1kcw	113	1fcp
17	1a7h	32	1ba7	83	2pab	94	1kit	115	1igt
18	1ha4	33	1jpc	84	1thw	96	1les	116	1hc9
19	2bb2	34	1k9b	86	1elp	97	1ofs	120	3jqo
20	1cbj	37	3gny	87	2rhe	98	1fep	121	3qtk
21	1m8u	72	1wnh	88	1bd7	99	2e8d		
22	1jbc	74	5dfr	89	1hk0	103	1nqh		
25	1ecz	75	1hk9	91	2yxf	106	1a0s		
27	1stp	78	2psg	92	1rav	107	1ku8		

set protein b_1 (PDB: 1n5u) was grouped into Cluster 2, then the first weight constant (c_1^p) of any new protein p would be predicted by BNN 2. If protein b_2 (PDB: 1lin) was in Cluster 4, the second weight constant (c_2^p) of any new protein p would be predicted by BNN 4, and so on.

This approach was intended to capture possible co-relationships between weight constants based on the structural similarities of the proteins (b_j) they are associated with in the original KCD method.

Mathematically, for a given training protein p with weight constants c_i^p ($i = 1...125$), and with b_i representing the i-th protein in the learning set L, the four output sets ($\overline{O}_k^p$) predicted by each BNN are defined as:

$$\begin{aligned} \overline{O}_1^p &= \{c_i^p \mid b_i \in P_1\} \\ \overline{O}_2^p &= \{c_i^p \mid b_i \in P_2\} \\ \overline{O}_3^p &= \{c_i^p \mid b_i \in P_3\} \\ \overline{O}_4^p &= \{c_i^p \mid b_i \in P_4\}. \end{aligned} \tag{3.1}$$

While the output targets are split, the input for each of the four BNNs remains the same: the four secondary structure features of the protein p.

$$\begin{aligned}
\overline{I}_1^p &= \{s_\alpha^p,\ s_\beta^p,\ s_{\text{coils}}^p,\ s_{\text{others}}^p\} \\
\overline{I}_2^p &= \{s_\alpha^p,\ s_\beta^p,\ s_{\text{coils}}^p,\ s_{\text{others}}^p\} \\
\overline{I}_3^p &= \{s_\alpha^p,\ s_\beta^p,\ s_{\text{coils}}^p,\ s_{\text{others}}^p\} \\
\overline{I}_4^p &= \{s_\alpha^p,\ s_\beta^p,\ s_{\text{coils}}^p,\ s_{\text{others}}^p\},
\end{aligned} \tag{3.2}$$

where s_α^p, s_β^p, s_{coils}^p, and s_{others}^p are the α, β, coils, and others, secondary structure content of the protein p, respectively.

Once we had the input and output data sets, we implemented each of the four BNNs using the PyTorch [57] framework. Every BNN model included a ReLU activation function for hidden layers, a SoftPlus activation function for the output layer, and the MSE loss function. Additionally, Bayesian optimization was implemented via Optuna [58] to find the best hyperparameters, with restrictions in the number of hidden layers (between 4 and 7), the number of neurons in each hidden layer (between 128 and 512), learning rate (between 1e-4 and 1e-2), Complexity cost weight (between 1e-7 and 1e-3), and the sample number (between 3 and 10), along with an early stopping patience of 25 epochs. All hyperparameter optimization was conducted with a package of 10,000 synthetic data as the training set and the KCD learning set as the validation set. The best hyperparameters found are listed in Table 3.7.

Table 3.7: Optimized hyperparameters for each of the four BNNs, found using Optuna.

Hyperparameter	**BNN 1**	**BNN 2**	**BNN 3**	**BNN 4**
Hidden Layers	7	7	6	5
Neurons in each layer	314, 330, 492, 494, 409, 283, 471	418, 510, 460, 321, 346, 216, 373	445, 294, 489, 400, 475, 442	420, 334, 490, 455, 408
Learning Rate	0.00311	0.00379	0.00450	0.00757
Sample Number	5	9	10	10
Complexity Cost Weight	4.55×10^{-5}	3.72×10^{-4}	1.06×10^{-6}	2.32×10^{-4}

Finally, each of the four BNNs was independently trained 5 times using its best hyperparameters. Each of these 5 runs used a different package of 10,000 synthetic data for training and was validated against the KCD learning set with early stopping. The model from the run with the lowest validation loss was selected as the final model for that BNN. This process of multiple independent runs was used to account for training stochasticity and ensure a robust final model was selected.

3.5 Integration with the KCD Model

Once the four Bayesian Neural Networks are trained, they are integrated into the KCD model to function as a single prediction engine. When a new protein p is entered for analysis, its four secondary structure features are fed as input to all four BNNs simultaneously.

Each BNN then predicts its own disjoint subset of the weight constants. These four partial outputs are concatenated, reassembling the complete 125-dimensional vector of predicted weight constants (c_i^p). This final vector is then used by the KCD model to perform the usual computations and calculate the protein's CD spectrum. This complete workflow is illustrated in Figure 3.6, and is referred to as KCD-AI.

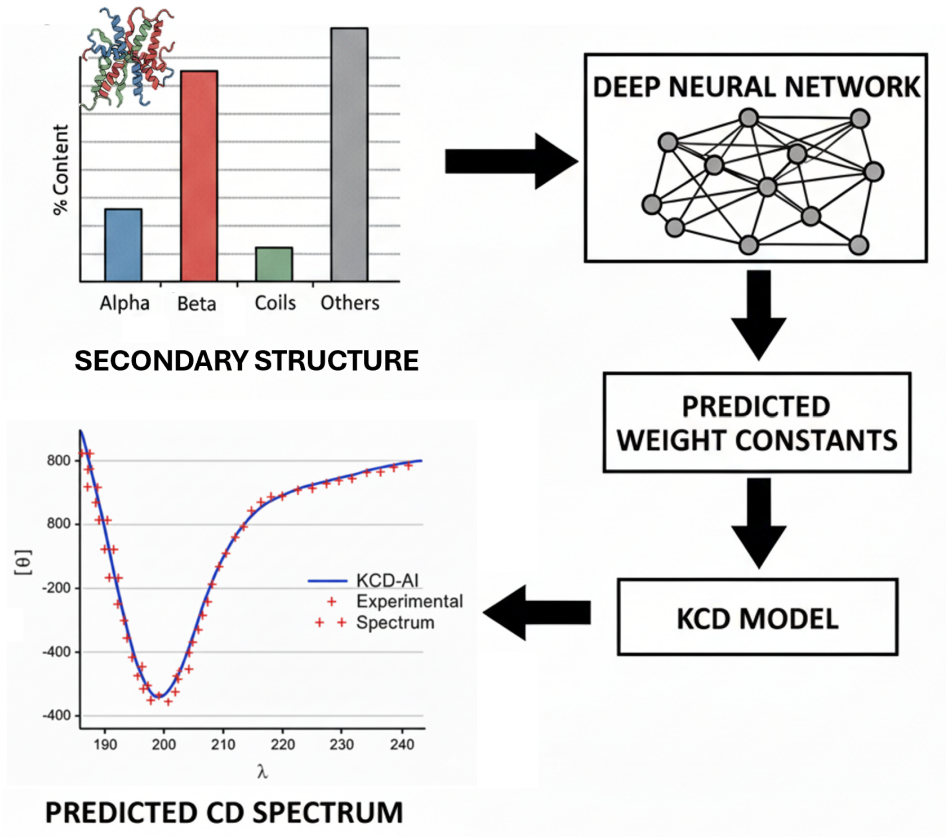


Figure 3.6: Conceptual workflow of the KCD-AI integration. (1) The secondary structure content of a protein (top left) is provided as input. (2) This is fed into the deep neural network framework (top right), which is represented here as a single block but conceptually consists of the four BNNs. (3) The BNNs output the 125 predicted weight constants. (4) The KCD model uses these constants to calculate the final CD spectrum (bottom left), which is then compared to the experimental spectrum.

Chapter 4

Results and Discussion

This chapter presents the results, organized into three main parts. First, we present the performance metrics of the deep neural networks developed during the training process. Next, we analyze the results from the integrated KCD-AI model, discussing its implications for protein studies and comparing its output to the original KCD model. Finally, the KCD-AI model is benchmarked against other state-of-the-art methods for computational CD spectra prediction.

4.1 Neural Network Performance

As mentioned in the preceding chapter, the Bayesian Neural Networks (BNNs) were trained using the Mean Squared Error (MSE) loss function. The loss was calculated at each epoch for both the training and validation sets.

To prevent overfitting, we implemented an early stopping mechanism based on the validation set. This procedure automatically halted a training run if the validation loss ceased to improve, even as the training loss continued to decrease. The training loss curve corresponding to the final, selected model is shown in Figure 4.1.

While validation loss was used to terminate individual runs, the final selection of the best-performing model was based on a different metric: the NAD (Normalized Absolute Deviation), eq. 2.19. For the most promising training runs, we computed the NAD between the full set of predicted constants and the targeted optimized constants for each protein in the validation set. The training that produced the best (lowest) average NAD was chosen as the final model, as shown in Figure 4.2.

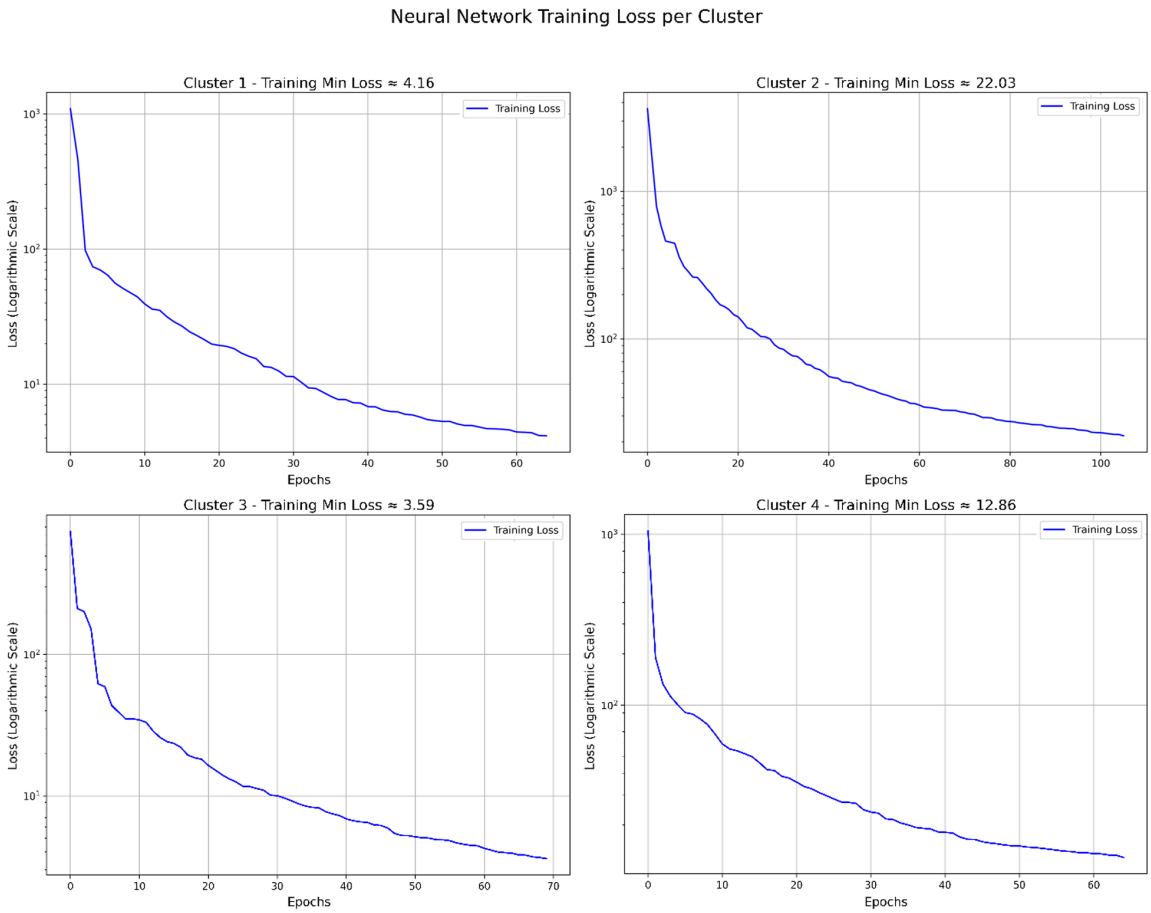


Figure 4.1: Training Loss of the four Bayesian Neural Networks (BNNs). The figure displays the best training loss curves for the four distinct BNNs. Each BNN was trained to predict a set of weight constants derived from a corresponding data cluster (Cluster 1-4). The x-axis represents the training epochs, and the y-axis shows the loss function value on a logarithmic scale. All four BNNs demonstrate successful convergence, with final minimum loss values of approximately 4.16 (BNN for Cluster 1), 22.03 (BNN for Cluster 2), 3.59 (BNN for Cluster 3), and 12.86 (BNN for Cluster 4).

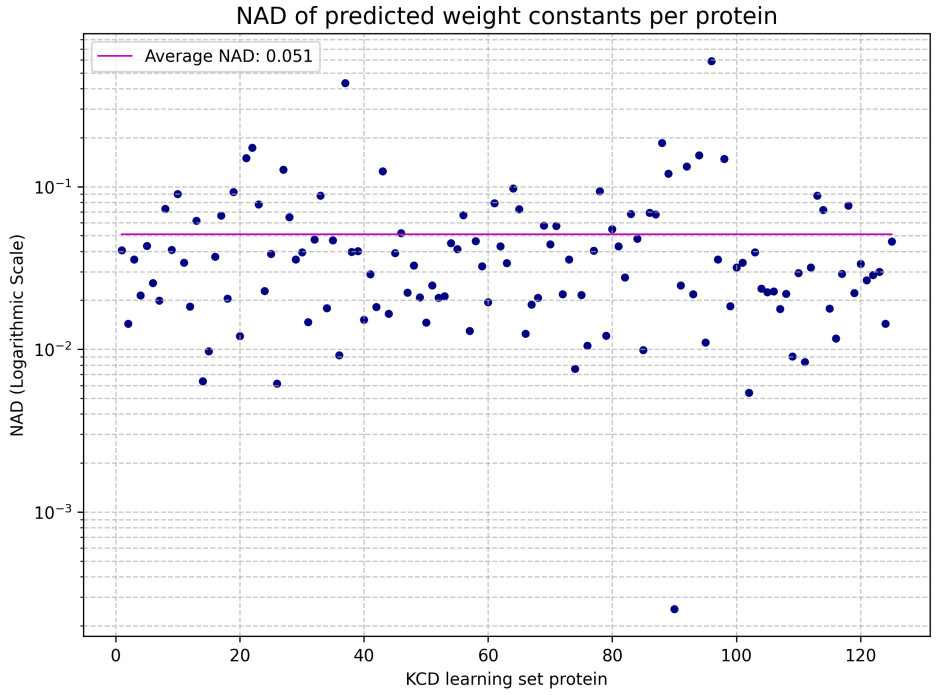


Figure 4.2: Normalized Absolute Deviation (NAD) of BNN-Predicted Weight Constants. The scatter plot illustrates the NAD for each protein from the KCD learning set (x-axis), which serves as the validation set in this context. Each point represents the deviation between the BNN's predicted weight constants and the corresponding optimized (ground-truth) constants for that protein. The y-axis is on a logarithmic scale, and the horizontal line shows the average NAD for all 125 proteins, calculated at 0.05 ± 0.07.

4.2 Impact on KCD Predictions

We first evaluated the model on the test set of 50 proteins (Table 3.2), comparing the performance of KCD-AI and KCD (Figure 4.3). We computed the average NAD for KCD as 0.25 ± 0.21 and for KCD-AI as 0.15 ± 0.14. This represents a 40% reduction in the average prediction error.

Additionally, the KCD-AI model provides a mean NAD and standard deviation for each protein, quantifying the reliability of its prediction. We note that proteins lacking significant α-helix content remain the most problematic, a common challenge for CD prediction models. However, KCD-AI successfully reduced the NAD for the majority of proteins compared to the original KCD.

In Figure 4.4, we present predictions for several proteins from the KCD learning set (Table 3.1). These are compared with the original KCD and the state-of-the-art method PDBMD2CD. As these proteins were part of the original KCD training, both KCD and KCD-AI are expected to perform well. In most cases, their predictions are very close, while the PDBMD2CD prediction is slightly more deviated. Nonetheless, KCD-AI demonstrates a more accurate prediction in several cases, such as for protein 1VJS, highlighting the optimization achieved by the BNNs.

Further examples are shown in Figure 4.5, which tests the models on proteins outside both the KCD learning and test sets (except for 1QFE). This analysis demonstrates the model's generalizability. In most cases, predictions from all three models are in good agreement with the experimental spectrum, with KCD-AI consistently providing a close or superior fit.

Finally, the power of KCD-AI in complex applications is shown in Figure 4.6. The model can be used to accurately assess computationally reconstructed protein structures, as demonstrated with 1K6J (NmrA). By comparing the predicted spectra from different reconstruction methods (Modeller, AF3, I-Tasser), one can select the structural model that best agrees with the experimental CD data. Furthermore, KCD-AI is sensitive enough to capture subtle conformational changes induced by the protein's environment, as shown for protein S100A12 in the presence of Zn^{2+} versus Ca^{2+} ions.

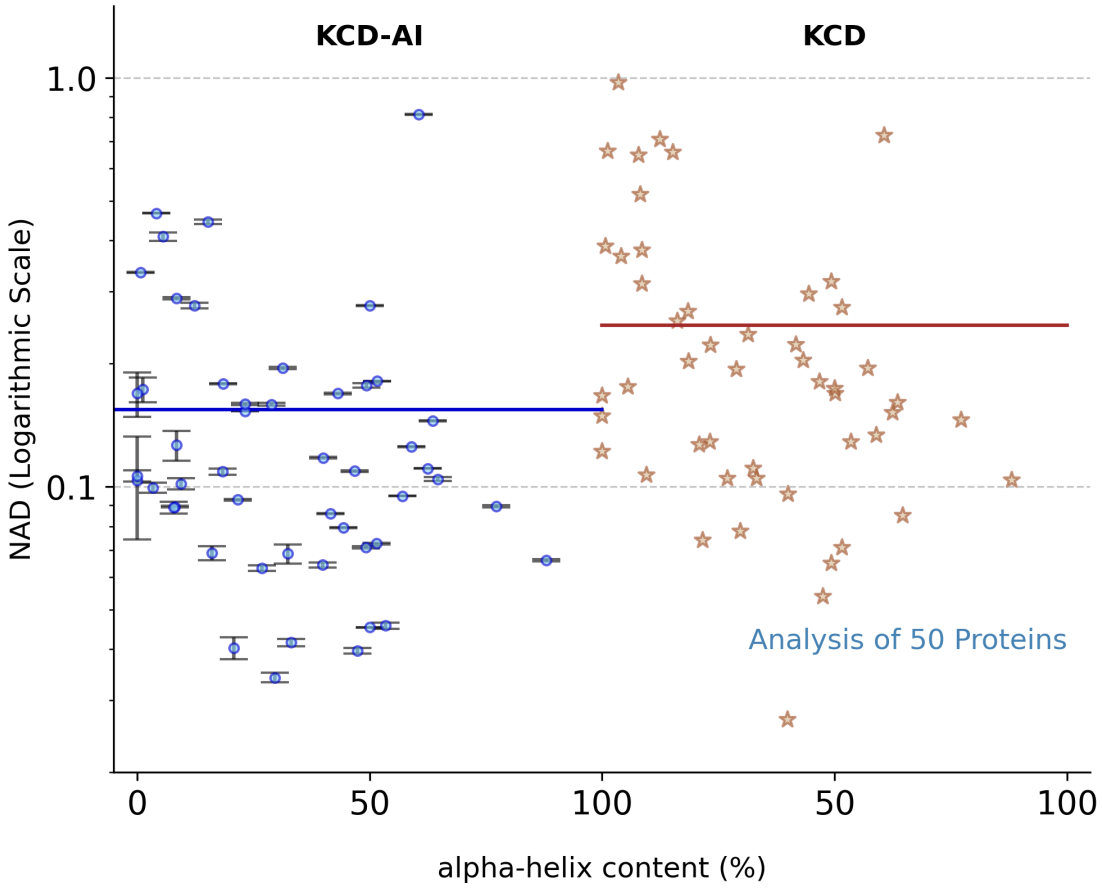


Figure 4.3: Comparison of the Normalized Absolute Deviation (NAD) for CD spectra predictions per protein in the test set, plotted as a function of protein α-helix content (%). The results for the KCD-AI model are shown as blue circles (mean) with their corresponding standard deviations (error bars). The results for the original KCD method are shown as sienna stars. The solid lines indicate the average NAD for each method: KCD-AI (blue, 0.15 ± 0.14) and KCD (sienna, 0.25 ± 0.21). The y-axis is on a logarithmic scale.

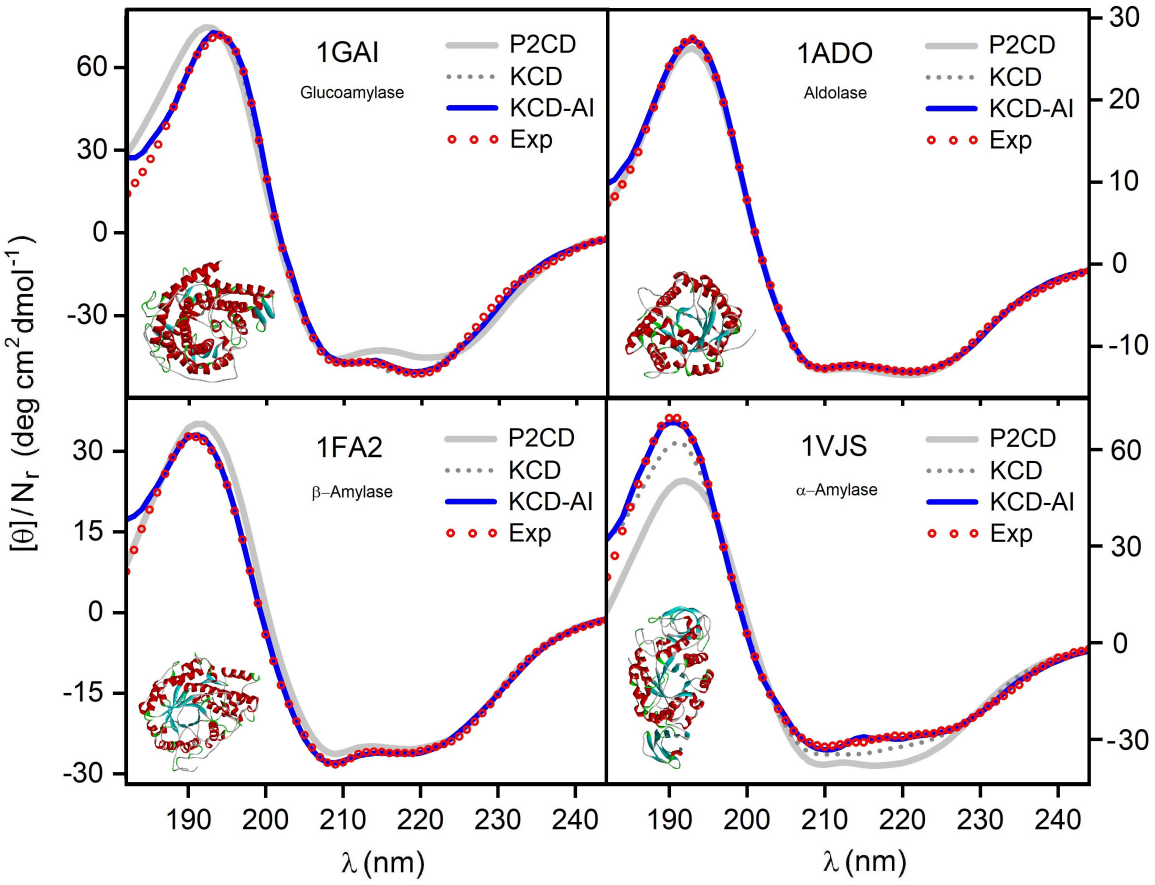


Figure 4.4: Comparison of predicted vs. experimental CD spectra for four proteins from the KCD learning set: 1GAI (Glucoamylase), 1ADO (Aldolase), 1FA2 (β-Amylase), and 1VJS (α-Amylase). Each panel plots the molar ellipticity per residue ($[\theta]/N_r$) as a function of wavelength (λ). The experimental SRCD spectrum [26] (Exp, red circles) is compared against three prediction methods: KCD-AI (solid blue line), the original KCD [33] (dotted gray line), and PDBMD2CD [30] (P2CD, solid gray line).

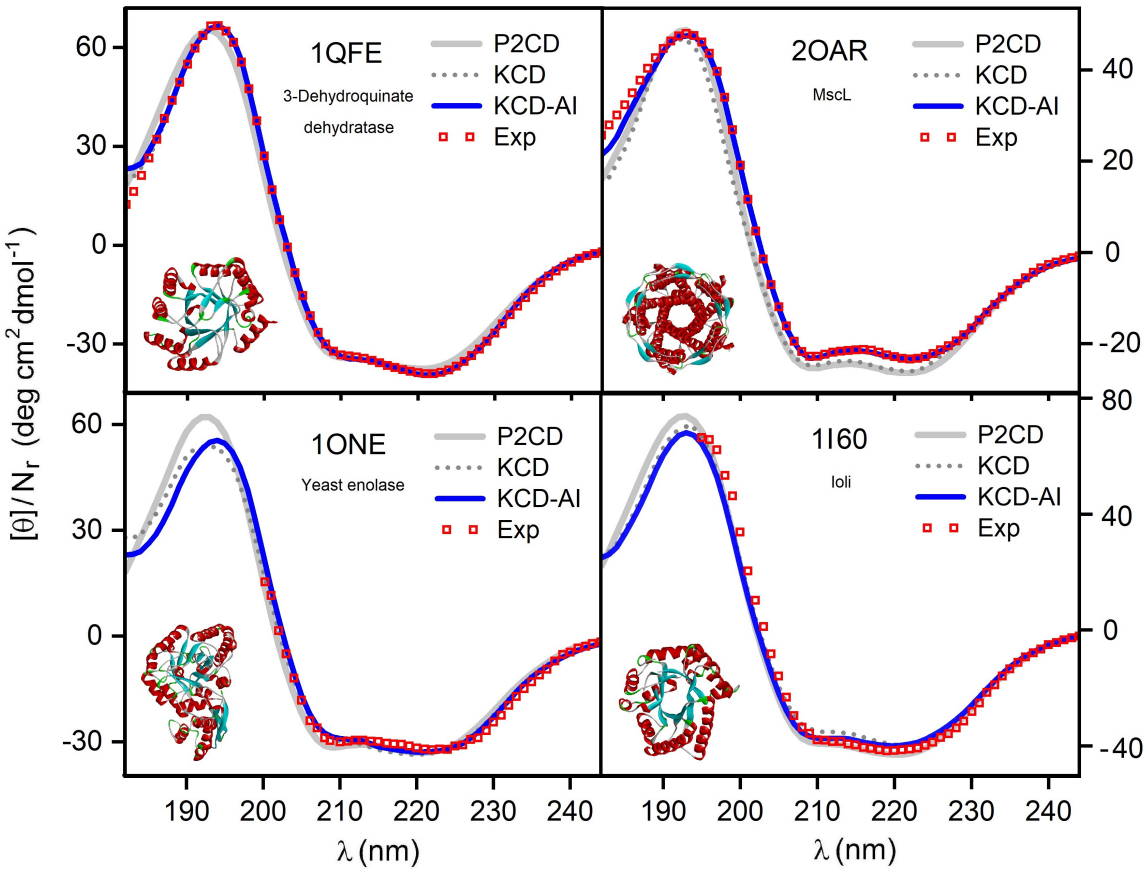


Figure 4.5: Comparison of predicted vs. experimental CD spectra for one protein from the KCD learning set: 1QFE (3-Dehydroquinate dehydratase). And 3 proteins outside the KCD learning set and the test set: 2OAR (MscL), 1ONE (Yeast enolase), and 1I60 (Ioli). Each panel plots the molar ellipticity per residue ($[\theta]/N_r$) as a function of wavelength (λ). The experimental SRCD spectrum [26] (Exp, red circles) is compared against three prediction methods: KCD-AI (solid blue line), the original KCD [33] (dotted gray line), and PDBMD2CD [30] (P2CD, solid gray line).

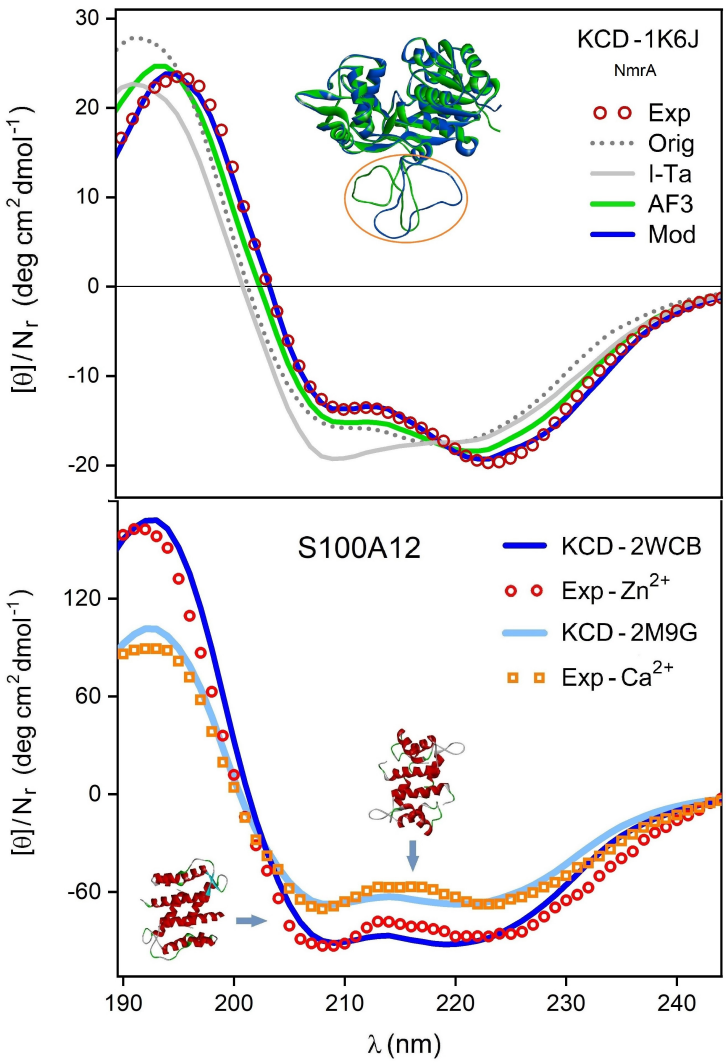


Figure 4.6: KCD-AI predictions for complex structural cases. Top panel: CD spectra for protein 1K6J (NmrA), which has a large missing loop (indicated by the orange-circled region in the structure). The experimental spectrum [26] (Exp, red circles) is compared with the KCD-AI prediction from the original, incomplete structure [13] (Orig, dotted gray line) and with predictions from three computationally-completed models: Modeller [40] (Mod, blue line), AlphaFold3 [59] (AF3, green line), and I-Tasser [60] (I-Ta, solid gray line). Bottom panel: Experimental and predicted spectra for protein S100A12, which exhibits different conformations when bound to different ions. The experimental spectrum for the Zn^{2+}-bound state [61] (Exp - Zn^{2+}, red circles) is compared with the KCD-AI prediction for its corresponding crystal structure, 2WCB [62] (KCD - 2WCB, dark blue line). The experimental spectrum for the Ca^{2+}-bound state [61] (Exp - Ca^{2+}, orange squares) is compared with the KCD-AI prediction for its corresponding crystal structure, 2M9G [63] (KCD - 2M9G, light blue line).

4.3 Comparison with State-of-the-Art Methods

In addition to comparing KCD-AI with the original KCD, we benchmarked it against several state-of-the-art CD spectra prediction tools: PDBMD2CD, SESCA, and DichroCalc. We computed the NAD for predictions from all methods across the 50-protein test set (Figure 4.7). The analysis found the following average NAD values:

- KCD-AI: 0.15 ± 0.14
- PDBMD2CD (P2CD): 0.25 ± 0.24
- SESCA: 0.28 ± 0.21
- DichroCalc (DC): 0.49 ± 0.40

KCD-AI demonstrated superior accuracy, achieving the lowest average prediction error on the test set.

In Figure 4.8, we show predictions for four representative proteins from the test set known to be challenging for CD prediction. Their secondary structure compositions are as follows:

- 1SR5 (Antithrombin): ∼16% α-helix, ∼26% β-sheet, ∼20% coil
- 3QTK (VEGF-A): ∼8% α-helix, ∼56% β-sheet, ∼16% coil
- 5A37 (α-Actin-2): ∼44% α-helix, ∼2% β-sheet, ∼26% coil
- 3GO0 (α-Defensin): ∼0% α-helix, ∼53% β-sheet, ∼17% coil

The protein 3GO0 is a particularly difficult case as it is composed entirely of D-amino acids. As shown in the figure, KCD-AI consistently provides superior performance, accurately predicting the spectra for these complex proteins. Notably, it is the only method to successfully predict the spectrum for the all-D-amino-acid protein, a case where all other benchmarked methods fail.

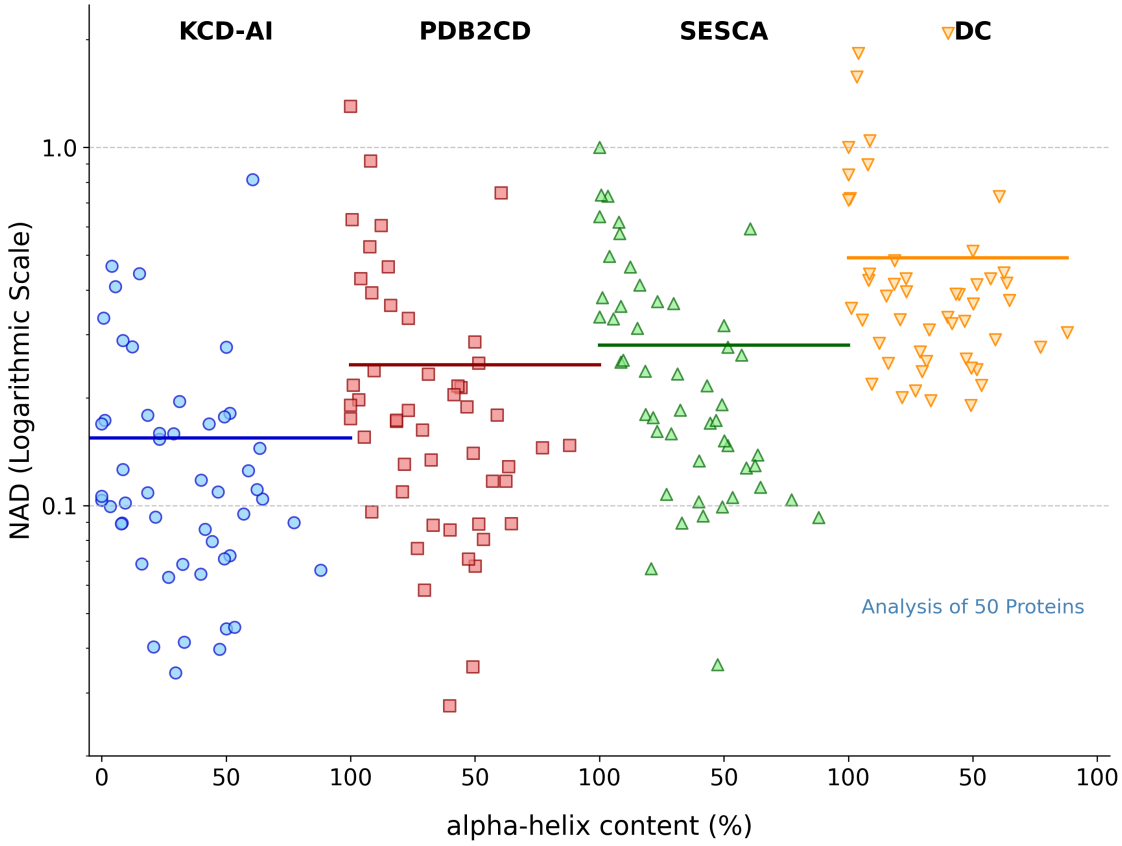


Figure 4.7: Comparison of the Normalized Absolute Deviation (NAD) for CD spectra predictions per protein in the test set, plotted as a function of protein α-helix content (%). The plot compares the performance of KCD-AI with three other state-of-the-art methods. KCD-AI results are shown as blue circles. PDBMD2CD [30] (PDB2CD) results are shown as red squares. SESCA [31] results are shown as green, upward-pointing triangles. DichroCalc [28] (DC) results are shown as orange, downward-pointing triangles. The solid lines indicate the average NAD for each method: KCD-AI (blue, 0.15 ± 0.14), PDB2CD (red, 0.25 ± 0.24), SESCA (green, 0.28 ± 0.21), and DC (orange, 0.49 ± 0.40). The y-axis is on a logarithmic scale.

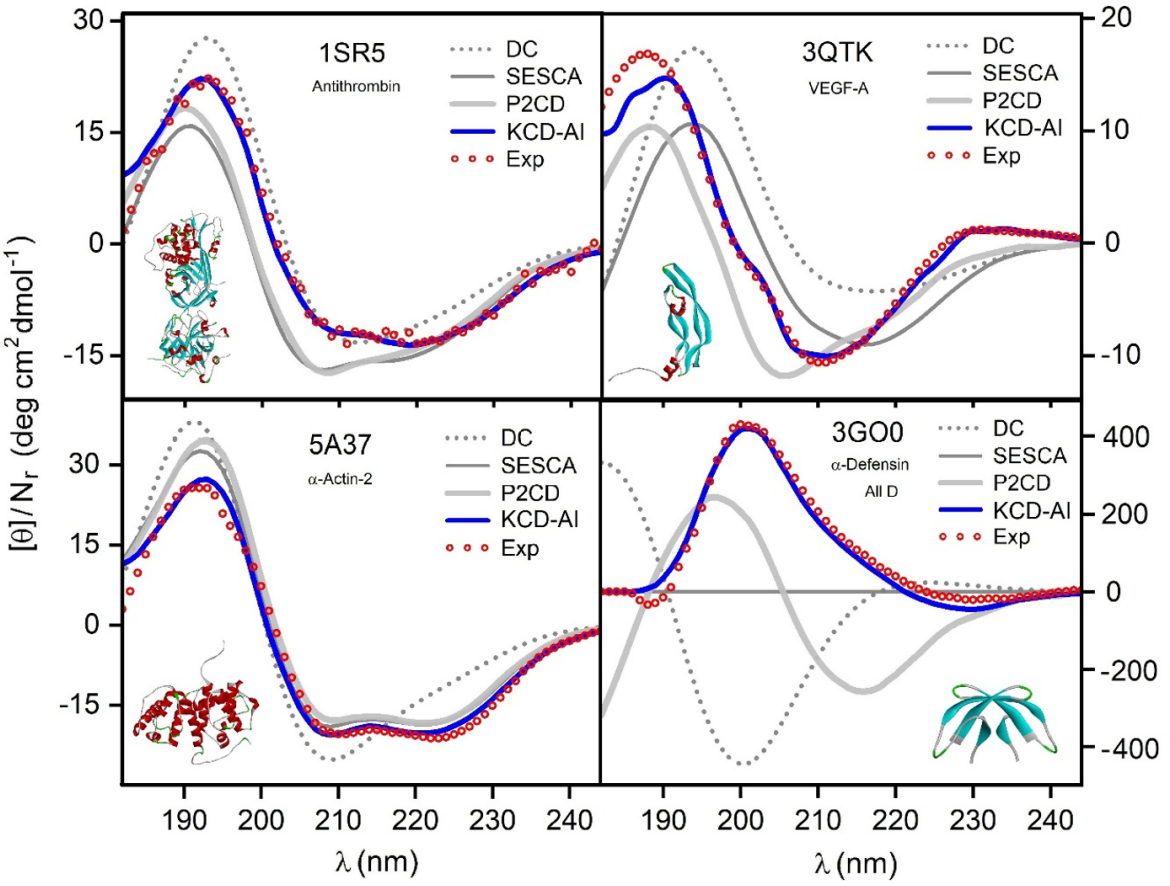


Figure 4.8: Comparison of predicted vs. experimental CD spectra for four proteins from the test set: 1SR5 (Antithrombin), 3QTK (VEGF-A), 5A37 (α-Actin-2), and the all-D-amino-acid protein 3GO0 (α-Defensin). Each panel plots the molar ellipticity per residue ($[\theta]/N_r$) as a function of wavelength (λ). The experimental SRCD spectrum [26] (Exp, red circles) is compared against four prediction methods: KCD-AI (solid blue line), PDBMD2CD [30] (P2CD, solid thick gray line), SESCA [31] (solid thin gray line), and DichroCalc [28] (DC, dotted gray line).

Chapter 5

Conclusions

We have successfully developed KCD-AI, the first model to integrate deep learning into the KCD framework. This refined computational model offers a powerful new tool for the structural characterization of proteins, improving predictive accuracy on average by 40% with respect to the original KCD.

A key methodological advance of KCD-AI is its use of Bayesian Neural Networks, which provide a quantitative measure of uncertainty (i.e., a standard deviation) for each prediction. This allows researchers to not only see the predicted spectrum but also to assess the model's "confidence", a feature absent in the original KCD and other deterministic methods.

While KCD-AI demonstrated significant improvement on average, its performance was not uniform. For a small subset of proteins, its predictive accuracy was comparable to the original KCD, and in a few isolated cases, its predictions were less accurate than the original method.

This highlights key areas for future research. First, a detailed analysis of these challenging cases is required to further refine the model. Second, expanding the training dataset is key to unlocking the full predictive power of our deep learning architecture. Finally, this work also served to identify fundamental limitations in the original KCD model itself. Addressing these underlying limitations will be crucial for building a stronger foundation for all future enhancements of the KCD method, including KCD-AI.

Bibliography

[1] P. Sweeney, H. Park, M. Baumann, J. Dunlop, J. Frydman, R. Kopito, A. McCampbell, G. Leblanc, A. Venkateswaran, A. Nurmi, and R. Hodgson, "Protein misfolding in neurodegenerative diseases: implications and strategies," *Translational Neurodegeneration*, vol. 6, p. 6, Mar. 2017.

[2] Q. Zhong, X. Xiao, Y. Qiu, Z. Xu, C. Chen, B. Chong, X. Zhao, S. Hai, S. Li, Z. An, and L. Dai, "Protein posttranslational modifications in health and diseases: Functions, regulatory mechanisms, and therapeutic implications," *MedComm*, vol. 4, no. 3, p. e261, 2023.

[3] "The Role of Protein Structure Analysis in the Success of Biopharmaceuticals," Sept. 2024.

[4] M. Batool, B. Ahmad, and S. Choi, "A structure-based drug discovery paradigm," *International Journal of Molecular Sciences*, vol. 20, no. 11, 2019.

[5] B. Alberts, R. Heald, A. Johnson, D. Morgan, M. Raff, K. Roberts, and P. Walter, *Molecular Biology of the Cell*. New York: W. W. Norton Company, 7th ed., 2022.

[6] H. Lodish, A. Berk, C. A. Kaiser, M. Krieger, A. Bretscher, H. Ploegh, K. C. Martin, M. B. Yaffe, and A. Amon, *Molecular Cell Biology*. W. H. Freeman, 9 ed., 2021.

[7] R. Zwanzig, A. Szabo, and B. Bagchi, "Levinthal's paradox.," *Proceedings of the National Academy of Sciences*, vol. 89, no. 1, pp. 20–22, 1992.

[8] C. B. Anfinsen, "Principles that govern the folding of protein chains," *Science*, vol. 181, no. 4096, pp. 223–230, 1973.

[9] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler,

T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis, "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, pp. 583–589, Aug. 2021.

[10] J. C. Kendrew, G. Bodo, H. M. Dintzis, R. G. Parrish, H. Wyckoff, and D. C. Phillips, "A Three-Dimensional Model of the Myoglobin Molecule Obtained by X-Ray Analysis," *Nature*, vol. 181, pp. 662–666, Mar. 1958.

[11] A. H. Moraes, D. M. Martins, and M. A. Chagas, *Exploring the Significance of Experimental and Computational Methods in Protein Structure Determination*, pp. 401–432. Cham: Springer Nature Switzerland, 2024.

[12] M. Adrian, J. Dubochet, J. Lepault, and A. W. McDowall, "Cryo-electron microscopy of viruses," *Nature*, vol. 308, pp. 32–36, Mar. 1984. Publisher: Nature Publishing Group.

[13] RCSB Protein Data Bank, "Rcsb pdb: Homepage." `https://www.rcsb.org/`.

[14] "Crystallography: Protein Data Bank," *Nature New Biology*, vol. 233, pp. 223–223, Oct. 1971. Publisher: Nature Publishing Group.

[15] H. M. Berman, J. Westbrook, Z. Feng, G. Gilliland, T. N. Bhat, H. Weissig, I. N. Shindyalov, and P. E. Bourne, "The Protein Data Bank," *Nucleic Acids Research*, vol. 28, pp. 235–242, Jan. 2000.

[16] RCSB Protein Data Bank, "Pdb statistics: Overall growth of released structures per year." `https://www.rcsb.org/stats/growth/growth-released-structures`, 2025. Accessed: 2025-08-10.

[17] N. J. Greenfield, "Using circular dichroism spectra to estimate protein secondary structure," *Nature Protocols*, vol. 1, pp. 2876–2890, Dec. 2006. Publisher: Nature Publishing Group.

[18] D. Corrêa and C. Ramos, "The use of circular dichroism spectroscopy to study protein folding, form and function," *African Journal of Biochemistry Research*, vol. 3, pp. 164–173, 04 2009.

[19] N. J. Greenfield, "Circular Dichroism Analysis for Protein-Protein Interactions," in *Protein-Protein Interactions: Methods and Applications* (H. Fu, ed.), pp. 55–77, Totowa, NJ: Humana Press, 2004.

[20] J. Kardos, M. P. Nyiri, Moussong, F. Wien, T. Molnár, N. Murvai, V. Tóth, H. Vadászi, J. Kun, F. Jamme, and A. Micsonai, "Guide to the structural characterization of protein aggregates and amyloid fibrils by CD spectroscopy," *Protein Science*, vol. 34, no. 3, p. e70066, 2025. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/pro.70066.

[21] R. W. Woody, "The development and current state of protein circular dichroism," *Biomedical Spectroscopy and Imaging*, vol. 4, no. 1, pp. 5–34, 2015.

[22] A. Rodger and D. Marshall, "Beginners guide to circular dichroism," *The Biochemist*, vol. 43, pp. 58–64, Mar. 2021.

[23] A. J. Miles, R. W. Janes, and B. A. Wallace, "Tools and methods for circular dichroism spectroscopy of proteins: a tutorial review," *Chemical Society Reviews*, vol. 50, no. 15, pp. 8400–8413, 2021.

[24] Protein Circular Dichroism Data Bank (PCDDB), "Homepage of the pcddb: a public repository for circular-dichroism spectra." `https://pcddb.cryst.bbk.ac.uk/`.

[25] Protein Circular Dichroism Data Bank, "PCDDB Statistics." `http://pcddb.cryst.bbk.ac.uk/`, 2025. Accessed: August 2025.

[26] S. G. Ramalli, A. J. Miles, R. W. Janes, and B. A. Wallace, "The PCDDB (Protein Circular Dichroism Data Bank): A Bioinformatics Resource for Protein Characterisations and Methods Development," *Journal of Molecular Biology*, vol. 434, p. 167441, June 2022.

[27] B. M. Bulheller and J. D. Hirst, "DichroCalc—circular and linear dichroism online," *Bioinformatics*, vol. 25, pp. 539–540, Feb. 2009.

[28] S. B. Jasim, Z. Li, E. E. Guest, and J. D. Hirst, "DichroCalc: Improvements in Computing Protein Circular Dichroism Spectroscopy in the Near-Ultraviolet," *Journal of Molecular Biology*, vol. 430, pp. 2196–2202, July 2018.

[29] L. Mavridis and R. W. Janes, "PDB2CD: a web-based application for the generation of circular dichroism spectra from protein atomic coordinates," *Bioinformatics*, vol. 33, pp. 56–63, Jan. 2017.

[30] E. D. Drew and R. W. Janes, "PDBMD2CD: providing predicted protein circular dichroism spectra from multiple molecular dynamics-generated protein structures," *Nucleic Acids Research*, vol. 48, pp. W17–W24, July 2020.

[31] G. Nagy, M. Igaev, N. C. Jones, S. V. Hoffmann, and H. Grubmüller, "SESCA: Predicting Circular Dichroism Spectra from Protein Molecular Structures," *Journal of Chemical Theory and Computation*, vol. 15, pp. 5087–5102, Sept. 2019. Publisher: American Chemical Society.

[32] D. Jacinto-Méndez, C. G. Granados-Ramírez, and M. D. Carbajal-Tinoco, "Kcd: Circular dichroism calculation for proteins." `https://kcd.cinvestav.mx/`, 2025.

[33] D. Jacinto-Méndez, C. G. Granados-Ramírez, and M. D. Carbajal-Tinoco, "KCD: A prediction web server of knowledge-based circular dichroism," *Protein Science*, vol. 33, no. 4, p. e4967, 2024. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/pro.4967.

[34] H. DeVoe, "Optical Properties of Molecular Aggregates. I. Classical Model of Electronic Absorption and Refraction," *The Journal of Chemical Physics*, vol. 41, pp. 393–400, July 1964.

[35] S. Superchi, E. Giorgio, and C. Rosini, "Structural determinations by circular dichroism spectra analysis using coupled oscillator methods: An update of the

applications of the DeVoe polarizability model," *Chirality*, vol. 16, no. 7, pp. 422–451, 2004. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/chir.20056.

[36] H. DeVoe, "Optical Properties of Molecular Aggregates. II. Classical Theory of the Refraction, Absorption, and Optical Activity of Solutions and Crystals," *The Journal of Chemical Physics*, vol. 43, pp. 3199–3208, Nov. 1965.

[37] C. G. Granados-Ramírez and M. D. Carbajal-Tinoco, "Knowledge-Based Atomic Polarizabilities Used to Model Circular Dichroism Spectra of Proteins," *The Journal of Physical Chemistry B*, vol. 126, pp. 80–92, Jan. 2022. Publisher: American Chemical Society.

[38] K. Ohta and H. Ishida, "Comparison among several numerical integration methods for kramers-kronig transformation," *Applied Spectroscopy*, vol. 42, pp. 952–957, 08 1988.

[39] A. Ponti, "Simulation of Magnetic Resonance Static Powder Lineshapes: A Quantitative Assessment of Spherical Codes," *Journal of Magnetic Resonance*, vol. 138, pp. 288–297, June 1999.

[40] D. S. BIOVIA, "Discovery studio visualizer," 2021. Version 2021, San Diego: Dassault Systèmes.

[41] D. Frishman and P. Argos, "Knowledge-based protein secondary structure assignment," *Proteins: Structure, Function, and Bioinformatics*, vol. 23, no. 4, pp. 566–579, 1995. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/prot.340230412.

[42] W. Kabsch and C. Sander, "Dictionary of protein secondary structure: Pattern recognition of hydrogen-bonded and geometrical features," *Biopolymers*, vol. 22, no. 12, pp. 2577–2637, 1983. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/bip.360221211.

[43] E. R. Kandel, J. H. Schwartz, and T. M. Jessell, *Principles of Neural Science*. New York: McGraw-Hill Education, 5 ed., 2013.

[44] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*. Springer Cham, 2 ed., 2023.

[45] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[46] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

[47] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, pp. 807–814, 2010.

[48] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.

[49] L. Lu, Y. Shin, Y. Su, and G. Em Karniadakis, "Dying relu and initialization: Theory and numerical examples," *Communications in Computational Physics*, vol. 28, p. 1671–1706, June 2020.

[50] H. Zheng, Z. Yang, W. Liu, J. Liang, and Y. Li, "Improving deep neural networks using softplus units," in *2015 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–4, 2015.

[51] A. Jadon, A. Patil, and S. Jadon, "A comprehensive survey of regression based loss functions for time series forecasting," *arXiv preprint arXiv:2211.02989*, 2022.

[52] T. Yu and H. Zhu, "Hyper-parameter optimization: A review of algorithms and applications," *CoRR*, vol. abs/2003.05689, 2020.

[53] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[54] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun, "Hands-on bayesian neural networks—a tutorial for deep learning users," *IEEE Computational Intelligence Magazine*, vol. 17, p. 29–48, May 2022.

[55] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. Cambridge, MA: MIT Press, 2012.

[56] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[57] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019.

[58] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, ACM, 2019.

[59] J. Abramson, J. Adler, J. Dunger, R. Evans, T. Green, A. Pritzel, O. Ronneberger, L. Willmore, A. J. Ballard, J. Bambrick, S. W. Bodenstein, D. A. Evans, C.-C. Hung, M. O'Neill, D. Reiman, K. Tunyasuvunakool, Z. Wu, A. Žemgulytė, and J. M. ... Jumper, "Accurate structure prediction of biomolecular interactions with alphafold 3," *Nature*, vol. 630, no. 8016, pp. 493–500, 2024.

[60] J. Yang and Y. Zhang, "I-tasser server: new development for protein structure and function predictions," *Nucleic Acids Research*, vol. 43, no. W1, pp. W174–W181, 2015.

[61] A. Garcia, J. Lopes, A. Costa-Filho, B. Wallace, and A. Araújo, "Membrane interactions of s100a12 (calgranulin c)," *PloS one*, vol. 8, p. e82555, 12 2013.

[62] O. V. Moroz, E. V. Blagova, A. J. Wilkinson, K. S. Wilson, and I. B. Bronstein, "The crystal structures of human s100a12 in apo form and in complex with zinc: New insights into s100a12 oligomerisation," *Journal of Molecular Biology*, vol. 391, no. 3, pp. 536–551, 2009.

[63] K. W. Hung, C. C. Hsu, and C. Yu, "Solution structure of human ca(2+)-bound s100a12," *Journal of Biomolecular NMR*, vol. 57, no. 3, pp. 313–318, 2013.